

Full Reference Manuscript: Exact Instantaneous Superposition in Nonlinear Dynamics

Reference Derivations for Tangent Space Linearization and Integral Accumulation

Dieter Olson

2026-01-01

Contents

Part I: Geometric Foundations and Local Dynamics	9
1 Introduction and Motivation	9
1.1 The Central Thesis	9
1.2 Structure of This Thesis	10
1.3 Scope and Prerequisites	11
1.3.1 Mathematical Prerequisites	11
1.3.2 Scope of Validity	12
2 Tangent Spaces and Exact Differentiation	13
2.1 Nonlinear Dynamics and Smoothness	13
2.2 Derivatives as Linear Maps	13
2.2.1 The Fréchet Derivative	13
2.2.2 Directional Derivatives	14
2.2.3 The Jacobian Matrices	15
2.3 Tangent Spaces: The Geometric Picture	15
2.3.1 Manifolds and Tangent Bundles	15
2.3.2 The Tangent Space Is Linear by Construction	16
2.3.3 Visualization: The Moving Tangent Plane	16
2.4 Example: The Simple Pendulum	16
2.4.1 System Dynamics	16
2.4.2 State-Space Representation	17
2.4.3 Jacobian Computation	17
2.4.4 Linearized Dynamics	18

2.4.5	Eigenanalysis	18
2.4.6	Where Nonlinearity Emerges	18
2.5	Exact Superposition in the Tangent Space	19
2.5.1	Superposition Properties	19
2.5.2	Counterfactual Decomposition	20
3	Time Evolution and Moving Tangent Spaces	21
3.1	Why Global Superposition Fails	21
3.1.1	Geometric Interpretation	21
3.1.2	Nonlinearity as Tangent Space Variation	22
3.2	Exactness in the Limit	22
3.2.1	Taylor Expansion Analysis	22
3.2.2	The Limit Argument	22
3.2.3	Variational Equation	23
4	Local Optimal Control: LQR and Lyapunov Stability	24
4.1	Connection to Linear Quadratic Regulation	24
4.1.1	The LQR Problem	24
4.1.2	What a Nonlinear LQR Design Establishes	24
4.1.3	Gain Scheduling Requires Its Own Check	25
4.2	Lyapunov Stability Analysis	25
4.2.1	Lyapunov's Indirect Method	25
4.2.2	Why This Works	25
4.2.3	Connection to Tangent Space Framework	26
4.3	Practical Implications	26
4.3.1	Design Philosophy	26
4.3.2	When Linear Techniques Suffice	27
4.3.3	When Nonlinear Techniques Required	27
5	Residuals, Reference Motion, and Nonlinearity	28
5.1	An Interaction Requires Four Comparable Trajectories	28
5.2	First Variations and the Mixed Second Variation	28
5.3	Transport Is Part of the Bound	29
5.4	What a Small Residual Does Not Establish	29
	Part II: Integral Superposition and Transport	30
6	Fundamentals of Integral Accumulation	30
7	Integration Preserves Linearity	30
7.1	From Infinitesimal to Finite Time	30
7.2	The Integration Theorem	30
7.3	What Superposes: Variations, Not Causes	31

8	State Transition Operator	31
8.1	Solution of the Variational Equation	31
8.2	Properties of Φ	32
8.3	Interpretation: Transport Then Accumulate	32
9	Time-Invariant Special Case	32
9.1	Constant Jacobian	32
9.2	Linear Time-Invariant (LTI) Systems	33
10	Variational Dynamics and Sensitivity Analysis	34
11	Forward Sensitivity	34
11.1	Sensitivity to Initial Conditions	34
11.2	Sensitivity to Inputs	34
11.3	Integrated Sensitivity	34
12	Adjoint (Costate) Dynamics	35
12.1	Backward Sensitivity	35
12.2	Adjoint Evolution Equation	35
12.3	Physical Interpretation	36
12.4	Duality: Forward and Backward	36
13	Physical Interpretations: Impulse and Work	37
14	Force-Momentum: Impulse Integral	37
14.1	Newton's Second Law	37
14.2	Impulse Definition	37
14.3	Superposition of Impulses	38
15	Power-Energy: Work Integral	38
15.1	Power and Work	38
15.2	Superposition of Work	39
16	General Principle: Integral Accumulation	39
17	Finite-Horizon Residuals and Error Quantification	41
17.1	Keep the Baseline and the Perturbation Definition Fixed	41
17.2	Interaction and Single-Trajectory Approximation Error	41
17.3	A Task-Based Interpretation for the Swing	42
Part III:	Hamiltonian Structure and Optimal Control	43
18	Variational Principles and Hamiltonian Formulation	43
18.1	Action Functional and Euler-Lagrange Equations	43
18.1.1	Lagrangian Mechanics	43

18.1.2	Hamiltonian Formulation	43
18.2	Costate as Sensitivity	44
18.2.1	Physical Meaning	44
18.2.2	Backward Integration	44
18.3	Detailed Variational Derivation	45
18.3.1	Calculus of Variations	45
18.3.2	Augmented Functional	45
18.3.3	First Variation	46
18.3.4	Optimality Conditions	46
18.4	Examples and Applications	47
18.4.1	Example 1: Minimum-Energy Problem	47
18.4.2	Example 2: Time-Optimal Control	48
18.4.3	Example 3: LQR as Hamiltonian System	48
18.5	Constrained Optimization	50
18.5.1	Inequality Constraints	50
18.5.2	State Constraints	50
18.6	Summary: Hamiltonian Framework	51
19	Numerical Implementation and Discrete-Time Systems	52
19.1	Discrete-Time Dynamics	52
19.1.1	Euler Discretization	52
19.1.2	Discrete Linearization	52
19.1.3	Per-Step Superposition	52
19.2	Discrete State Transition	53
19.2.1	Recursive Computation	53
19.3	Higher-Order Integration Schemes	53
19.3.1	Runge-Kutta Methods	53
19.3.2	Adaptive Step Size	54
19.3.3	Implicit Methods	54
19.4	Convergence Analysis	55
19.4.1	Local vs. Global Error	55
19.4.2	Theorem: Convergence of Euler's Method	55
19.4.3	Higher-Order Methods	55
19.5	Numerical Stability	56
19.5.1	Stability Region	56
19.5.2	Stiffness and Timestep Selection	56
19.6	Error Accumulation and Bounds	56
19.6.1	Accumulation Over Horizon	56
19.6.2	Discrete Tangent Space Errors	57
19.7	Practical Implementation	57
19.7.1	Pseudo-Code: RK4 Integration	57
19.7.2	Jacobian Computation	58
19.8	Comparison of Methods	58

19.9 Summary: Discrete Implementation	59
20 Control Synthesis via DDP, iLQR, and MPC	60
20.1 Differential Dynamic Programming (DDP)	60
20.1.1 Algorithm Overview	60
20.1.2 Connection to Integral Superposition	60
20.2 Iterative LQR (iLQR)	60
20.3 Model Predictive Control (MPC)	61
20.4 Detailed DDP Derivation	61
20.4.1 Cost-to-Go Recursion	61
20.4.2 Quadratic Approximation	61
20.4.3 Action-Value Function (Q-Function)	62
20.4.4 Optimal Control Update	63
20.4.5 Value Function Update	63
20.5 Complete DDP Algorithm	63
20.6 Iterative LQR (iLQR) Variant	64
20.7 Convergence Requires More Than Positive Input Curvature	64
20.8 Constrained MPC Extension	65
20.8.1 Handling Control Bounds	65
20.8.2 State Constraints	65
20.9 Model Predictive Control (MPC)	66
20.9.1 Receding Horizon Strategy	66
20.9.2 Stability Guarantees	66
20.10 Computational Complexity and Timing Budgets	67
20.11 Robust and Stochastic Extensions	67
20.11.1 Robust MPC	67
20.11.2 Stochastic MPC	67
20.12 Summary: DDP/iLQR/MPC	68
21 Case Studies and Worked Examples	69
21.1 Case Study 1: Spacecraft Rendezvous	69
21.1.1 Problem Formulation	69
21.1.2 Dynamics Implementation	70
21.1.3 iLQR Solution	71
21.1.4 Results	73
21.2 Case Study 2: Planar Robot Arm	73
21.2.1 Problem Formulation	73
21.2.2 Implementation	74
21.2.3 DDP Solution	76
21.3 Case Study 3: Quadrotor Stabilization	76
21.3.1 Problem Formulation	76
21.3.2 Linearization Challenge	77
21.3.3 DDP Implementation Sketch	77

21.4 Comparative Analysis	78
21.5 Reference Implementation Notes	78
22 Conclusion and Future Directions	79
23 Summary of Key Results	79
23.1 Part I: Local Exactness	79
23.2 Part II: Integral Accumulation	79
23.3 Part III: Optimal Control Applications	79
24 Conceptual Contributions	80
24.1 Reframing Linearization	80
24.2 The Superposition Spectrum	80
24.3 Geometric vs. Analytic Perspectives	80
25 Practical Impact	81
25.1 Design Methodology	81
25.2 Analysis Tools	81
25.3 Software Implementation	81
26 Limitations and Open Questions	81
26.1 Scope Limitations	81
26.2 Computational Challenges	82
26.3 Fundamental Questions	82
27 Future Work	82
27.1 Theoretical Extensions	82
27.2 Practical Applications	82
27.3 Pedagogical Development	83
28 Final Reflection	83
References	84
Appendices	85
29 Appendix a: Mathematical Prerequisites	85
30 Appendix B: Proofs and Derivations	85
31 B.1 A Finite-Difference Bound From the Endpoint Map	85
32 B.2 Intrinsic Curvature Is a Different Quantity	86
33 B.3 Integration Error and Linearization Error	86

34 Appendix C: Notation Guide and Glossary	86
35 Primary Notation	86
36 Glossary	87
37 Appendix D: Code Repository Index	87

Reference Status

This is the single-file full reference manuscript for the tangent-space material. It is retained for derivations, appendices, and historical traceability. The canonical rendered reading path is the compact seven-part [Tangent-Space Series](#), which now carries the contraction, hybrid, and residual-aware topics in shorter source-backed form.

Thesis Abstract

Nonlinear dynamical systems are commonly characterized as systems where the superposition principle does not hold. This thesis challenges that characterization, arguing that it obscures a more precise mathematical fact: **at a fixed nominal state and input, the differential mapping from perturbations to state-rate variations is linear; for a control-affine model, finite input increments also map linearly into instantaneous rate increments at fixed state and mode, and the associated variational equations accumulate perturbations linearly along a chosen reference trajectory.** We do not claim that motion trajectories superpose—nonlinear flows are generally not additive. Superposition here refers to the instantaneous tangent-space structure.

We develop this claim across three interconnected frameworks. Part I establishes that linearization is not an approximation but the exact infinitesimal structure of smooth dynamics—the derivative of a vector field is, by definition, a linear map operating on a linear tangent space. Part II shows how the linear variational system propagates perturbations along a reference trajectory. Part III connects these geometric foundations to optimal control through Hamiltonian structures and adjoint dynamics.

This unified perspective helps explain why linear control techniques can remain useful for nonlinear plants and why linearization can yield rigorous local conclusions once the hypotheses are stated explicitly. We illustrate with worked examples including the pendulum, demonstrate connections to LQR and Lyapunov stability, and delineate scope of validity. Throughout, we distinguish between local exactness (in tangent spaces) and finite-scale approximation limits (due to curvature, mode changes, and model scope), with residuals treated as geometric terms that must still be bounded in context.

Keywords: Tangent spaces, linearization, variational dynamics, state transition operator, Hamiltonian mechanics, optimal control, differential geometry

i A Note on “Exactness”

Throughout this thesis, “exact” means *exact in the infinitesimal limit*. The tangent space captures the system’s instantaneous behavior with complete mathematical fidelity — this is the definition of the derivative, not an approximation. For finite perturbations, second-order residuals arise with magnitude $O(\|\delta x\|^2)$. [Part IV: Residual-Aware Control](#) discusses how such residual terms might be bounded and used in algorithm design when the needed smoothness and estimation assumptions hold.

Part I: Geometric Foundations and Local Dynamics

1 Introduction and Motivation

1.1 The Central Thesis

Nonlinear dynamics are often described as systems in which superposition does not apply. Students of differential equations learn early that while linear systems permit the combination of solutions—if $x_1(t)$ and $x_2(t)$ are solutions, then so is $\alpha x_1(t) + \beta x_2(t)$ —nonlinear systems deny this convenience. The narrative typically stops there, leaving the impression that nonlinearity and superposition are fundamentally incompatible.

This statement, while true in a global sense, obscures a more precise and mathematically exact fact:

! Central Claim

At each fixed state and fixed contact/constraint mode, nonlinear systems admit exact linear structure in the instantaneous tangent-space mapping; along a chosen reference trajectory, the associated variational equations accumulate perturbations linearly.

⚠ Scope: No Trajectory Superposition

We will never claim that *motion trajectories* superpose in nonlinear systems. Trajectory-level additivity is not implied—nonlinear flows generally do not superpose, as established in standard treatments (e.g., Slotine & Li, *Applied Nonlinear Control*, §3.1). Superposition here refers to **instantaneous** tangent-space objects (velocity and acceleration) and, in mechanics, to the **linearity of accelerations with respect to applied generalized forces** at a frozen state and fixed contact mode. The tangent space **moves with the state**, so finite-time trajectories are not additive.

i Terminology: “Tangent Hyperplane” as Affine Subspace

The term “tangent hyperplane” is used colloquially throughout this series to evoke the geometric intuition of a flat structure touching a curved space. More precisely, the relevant geometric object is an **affine subspace** of the tangent space. When the input distribution $\text{span}\{g_i(x)\}$ has codimension greater than 1, the reachable set within the tangent space is an affine subspace, not strictly a hyperplane (which implies codimension 1). We retain the term for accessibility while noting this distinction for mathematical precision.

The purpose of this thesis is to develop that statement rigorously. The differential-geometric core is not heuristic, but its mechanics interpretations still depend on the model class, smoothness assumptions, and contact mode being held fixed. Within that scope, the argument rests on the same calculus and variational machinery used throughout modern nonlinear dynamics.

What This Is **NOT**

This is not “another linearization approximation paper.”

We are **not** claiming that linearization is “good enough” for nonlinear systems. We are claiming something much stronger and more precise:

- **The tangent space is exact** (not approximate) infinitesimal structure—it is the Fréchet derivative, which is the unique best linear approximation by definition
- **Superposition is exact** (not approximate) in tangent spaces because tangent spaces are vector spaces
- **Finite perturbations require an approximation** even in a fixed Euclidean coordinate space; distinguish nonlinear interaction from integration error and model error

Key distinction: Standard linearization says “we accept error for tractability.” This thesis says “we exploit exact local geometry and quantify transport costs.” The exactness comes from **geometry**, not from “being close to the operating point.”

This perspective has profound implications:

1. It explains why **linear control techniques** such as LQR and pole placement succeed in practice, even when applied to inherently nonlinear plants
2. It clarifies why **stability analysis via linearization** (Lyapunov’s indirect method) yields rigorous conclusions about nonlinear behavior
3. It provides the **geometric foundation** for modern nonlinear control theory, where trajectories are curves on manifolds with tangent spaces attached at each point
4. It establishes that **interaction residuals** can occur in a perfect nonlinear model, while measured residuals can additionally contain modeling and numerical error

1.2 Structure of This Thesis

This thesis develops the “exact superposition in the infinitesimal” framework across three interconnected parts:

Part I: Geometric Foundations and Local Dynamics

- Chapter 1: Tangent spaces and the exact nature of differentiation
- Chapter 2: Time evolution through moving tangent spaces

- Chapter 3: Local optimal control in tangent space (LQR, Lyapunov)
- Chapter 4: Residuals of the nonlinear input-to-flow map

Part II: Integral Superposition and Transport

- Chapter 5: Integration preserves linear accumulation of variations
- Chapter 6: State transition operators and variational dynamics
- Chapter 7: Physical interpretations (impulse, work) as integral consequences
- Chapter 8: Global residuals and error quantification

Part III: Hamiltonian Structure and Optimal Control

- Chapter 9: Variational principles and Hamiltonian formulation
- Chapter 10: Adjoint (costate) dynamics as backward sensitivity integration
- Chapter 11: Control synthesis via DDP, iLQR, and MPC
- Chapter 12: Worked examples and case studies

Each part builds on the previous, progressing from pure geometric foundations through integral accumulation to practical optimal control applications.

1.3 Scope and Prerequisites

1.3.1 Mathematical Prerequisites

This thesis assumes familiarity with:

- **Multivariable calculus:** Partial derivatives, gradients, Jacobians
- **Linear algebra:** Vector spaces, linear maps, eigenvalues
- **Ordinary differential equations:** Existence and uniqueness theorems
- **Basic functional analysis:** Normed spaces, limits, $o(\cdot)$ and $O(\cdot)$ notation

The thesis introduces, as needed:

- Differential geometry concepts (manifolds, tangent bundles)
- Variational calculus (Euler-Lagrange equations, Hamiltonian mechanics)
- Control theory foundations (state-space representation, LQR, stability)

1.3.2 Scope of Validity

The mathematical framework developed here applies to:

Smooth nonlinear systems: Vector fields $f(x, u)$ that are continuously differentiable (C^1)

This includes:

- Mechanical systems (Newtonian, Lagrangian, Hamiltonian)
- Robotic manipulators and vehicles
- Aerospace systems (aircraft, spacecraft)
- Chemical process control
- Power systems and electrical machines

The framework requires modification or does not apply to:

Discontinuous systems: Switching controllers, relay systems, variable-structure control

Impact dynamics: Collisions, rigid body contact with instantaneous velocity changes

Coulomb friction: Stick-slip dynamics with set-valued force laws

Hybrid systems: Discrete state transitions combined with continuous dynamics

State constraints: Hard boundaries with inequality constraints

For these cases, generalizations exist (differential inclusions, Filippov solutions, complementarity systems, hybrid automata) but lie outside this thesis scope.

2 Tangent Spaces and Exact Differentiation

2.1 Nonlinear Dynamics and Smoothness

Consider a nonlinear dynamical system described by the ordinary differential equation:

$$\dot{x} = f(x, u), \quad x \in \mathbb{R}^n, \quad u \in \mathbb{R}^m \quad (1)$$

where:

- x represents the **state vector** (positions, velocities, temperatures, concentrations, etc.)
- u represents the **control input** or external forcing
- $f : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}^n$ is the **vector field** encoding system physics

The critical assumption is that f is **continuously differentiable** (C^1) in both arguments. This means:

1. f is continuous
2. All partial derivatives $\frac{\partial f_i}{\partial x_j}$ and $\frac{\partial f_i}{\partial u_k}$ exist
3. These partial derivatives are themselves continuous functions

i Why C^1 Smoothness Matters

This assumption is minimal—it represents the bare mathematical requirement for derivatives to exist—yet physically reasonable. Virtually all systems derived from classical mechanics satisfy this smoothness condition. The laws of physics, expressed through Newton’s equations, Lagrangian mechanics, or thermodynamic principles, generically produce smooth vector fields.

What does smoothness buy us? It guarantees that at every point in state-input space, the vector field f has a well-defined derivative. This derivative is a **linear map**—and it is through this linear map that superposition re-enters the picture.

2.2 Derivatives as Linear Maps

2.2.1 The Fréchet Derivative

In single-variable calculus, the derivative at a point is a number—the slope of the tangent line. In multivariable calculus, this generalizes: the derivative of a vector-valued function is not a number but a **linear map**. This is the Fréchet derivative, the mathematically precise formulation of differentiation in higher dimensions.

For our vector field f , the Jacobian matrix $A = \frac{\partial f}{\partial x}(x_0, u_0)$ is defined as the unique linear map satisfying:

$$f(x_0 + \delta x, u_0) = f(x_0, u_0) + A \cdot \delta x + o(\|\delta x\|) \quad (2)$$

The notation $o(\|\delta x\|)$ denotes terms that vanish faster than $\|\delta x\|$ as $\|\delta x\| \rightarrow 0$. Precisely:

$$\lim_{\|\delta x\| \rightarrow 0} \frac{\|f(x_0 + \delta x, u_0) - f(x_0, u_0) - A \cdot \delta x\|}{\|\delta x\|} = 0 \quad (3)$$

This captures what we mean by “the error becomes negligible”—the remainder term becomes arbitrarily small relative to the perturbation size.

! Exactness in the Limit

Equation Equation 2 states that near the point (x_0, u_0) , the nonlinear function f is **exactly equal** to a linear function plus terms that become arbitrarily small relative to the perturbation. This is not an approximation we impose for convenience—it is the definition of the derivative.

2.2.2 Directional Derivatives

An equivalent, perhaps more intuitive, definition uses directional derivatives. For any direction $h \in \mathbb{R}^n$:

$$A \cdot h = \lim_{t \rightarrow 0} \frac{f(x_0 + th, u_0) - f(x_0, u_0)}{t} \quad (4)$$

This limit computes the rate of change of f as we move from x_0 in the direction h . The crucial point: **this rate of change depends linearly on the direction h** .

- If we double the direction vector, the rate of change doubles: $A \cdot (2h) = 2(A \cdot h)$ (homogeneity)
- If we add two direction vectors, the rates of change add: $A \cdot (h_1 + h_2) = A \cdot h_1 + A \cdot h_2$ (additivity)

These are the defining properties of a **linear map**. This is not an approximation or simplification—it is a mathematical theorem following from the definition of the derivative and the smoothness of f .

2.2.3 The Jacobian Matrices

At a fixed operating point (x_0, u_0) , we compute two derivative matrices:

$$A = \left. \frac{\partial f}{\partial x} \right|_{(x_0, u_0)} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \cdots & \frac{\partial f_n}{\partial x_n} \end{bmatrix}_{(x_0, u_0)} \quad (5)$$

$$B = \left. \frac{\partial f}{\partial u} \right|_{(x_0, u_0)} = \begin{bmatrix} \frac{\partial f_1}{\partial u_1} & \cdots & \frac{\partial f_1}{\partial u_m} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial u_1} & \cdots & \frac{\partial f_n}{\partial u_m} \end{bmatrix}_{(x_0, u_0)} \quad (6)$$

These matrices have specific interpretations:

- A : State Jacobian ($n \times n$)—how each component of $\dot{x}$ depends on each state variable
- B : Input Jacobian ($n \times m$)—how $\dot{x}$ responds to control inputs

Together, (A, B) define the **tangent space linearization** at that point.

2.3 Tangent Spaces: The Geometric Picture

2.3.1 Manifolds and Tangent Bundles

To understand “tangent space linearization,” we invoke differential geometry. A **smooth manifold** is a space that locally resembles Euclidean space—think of Earth’s surface, which is curved globally but flat in small neighborhoods.

At each point x_0 on a manifold $\mathcal{M}$, there exists a **tangent space** $T_{x_0}\mathcal{M}$: the collection of all possible velocity vectors for curves passing through that point. For a sphere, the tangent space at any point is a flat plane touching the sphere at that point.

For dynamical systems with states in $\mathbb{R}^n$, the tangent space at x_0 is itself an n -dimensional vector space. This tangent space is where linearization lives.

2.3.2 The Tangent Space Is Linear by Construction

When we perturb the state by δx and the input by δu , the resulting perturbation to $\dot{x}$ is governed by:

$$\delta \dot{x} = A \delta x + B \delta u \quad (7)$$

This equation deserves careful attention. It states that **in the tangent space at a fixed state, the dynamics are exactly linear**. The perturbation $\delta \dot{x}$ depends linearly on δx and δu , with coefficients given by the Jacobian matrices. Note that the **first-order differential** (linearization) is exact at the point; finite steps accumulate higher-order terms.

! Not an Approximation

This is not an approximation of the nonlinear system—it **is** the differential structure of the system at that state. The linearization captures the infinitesimal behavior exactly, in the same sense that the derivative of a curve gives its exact instantaneous velocity.

2.3.3 Visualization: The Moving Tangent Plane

Imagine a hiker walking along a mountain ridge:

- At each point, the ground is **locally flat** (the tangent plane)
- But the tangent plane **rotates and tilts** as the hiker moves
- The mountain is **curved** even though every tangent plane is **flat**

This is precisely the geometric picture for nonlinear dynamics:

- The state space is a **manifold** (possibly curved, complex topology)
- At each point there is a **tangent space** where dynamics are exactly linear
- As a trajectory evolves, it passes through a **continuous family of tangent spaces**

2.4 Example: The Simple Pendulum

2.4.1 System Dynamics

Consider the simple pendulum—a mass hanging from a rigid rod of length L , swinging under gravity. Let:

- θ : angle from vertical (downward)
- $\omega = \dot{\theta}$: angular velocity
- g : gravitational acceleration

Newton's second law yields:

$$\ddot{\theta} = -\frac{g}{L} \sin \theta \quad (8)$$

This is manifestly nonlinear: the restoring torque depends on $\sin \theta$, not θ itself. The $\sin$ function prevents global superposition— $\sin(\theta_1 + \theta_2) \neq \sin \theta_1 + \sin \theta_2$.

2.4.2 State-Space Representation

Rewrite as a first-order system. Define state vector $x = [\theta, \omega]^T$:

$$\dot{x} = f(x) = \begin{bmatrix} \omega \\ -\frac{g}{L} \sin \theta \end{bmatrix} \quad (9)$$

The function f is smooth (infinitely differentiable), so our theory applies.

2.4.3 Jacobian Computation

At the equilibrium $x_0 = [0, 0]^T$ (pendulum hanging straight down at rest):

$$A = \left. \frac{\partial f}{\partial x} \right|_{x_0} = \begin{bmatrix} \frac{\partial f_1}{\partial \theta} & \frac{\partial f_1}{\partial \omega} \\ \frac{\partial f_2}{\partial \theta} & \frac{\partial f_2}{\partial \omega} \end{bmatrix}_{x_0} \quad (10)$$

Computing entries:

- $\frac{\partial f_1}{\partial \theta} = \frac{\partial \omega}{\partial \theta} = 0$
- $\frac{\partial f_1}{\partial \omega} = \frac{\partial \omega}{\partial \omega} = 1$
- $\frac{\partial f_2}{\partial \theta} = \frac{\partial}{\partial \theta} \left(-\frac{g}{L} \sin \theta \right) = -\frac{g}{L} \cos \theta \big|_{\theta=0} = -\frac{g}{L}$
- $\frac{\partial f_2}{\partial \omega} = 0$

Therefore:

$$A = \begin{bmatrix} 0 & 1 \\ -\frac{g}{L} & 0 \end{bmatrix} \quad (11)$$

2.4.4 Linearized Dynamics

The tangent space dynamics are:

$$\delta \dot{x} = A \delta x \quad \Rightarrow \quad \begin{bmatrix} \delta \dot{\theta} \\ \delta \dot{\omega} \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -\frac{g}{L} & 0 \end{bmatrix} \begin{bmatrix} \delta \theta \\ \delta \omega \end{bmatrix} \quad (12)$$

From the second component: $\delta \dot{\omega} = -\frac{g}{L} \delta \theta$. Since $\delta \omega = \delta \dot{\theta}$:

$$\delta \ddot{\theta} = -\frac{g}{L} \delta \theta \quad (13)$$

This is the **harmonic oscillator equation**. Every physics student knows this as the “small angle approximation”—but our analysis reveals it as something more fundamental.

2.4.5 Eigenanalysis

The eigenvalues of A are found from $\det(A - \lambda I) = 0$:

$$\det \begin{bmatrix} -\lambda & 1 \\ -\frac{g}{L} & -\lambda \end{bmatrix} = \lambda^2 + \frac{g}{L} = 0 \quad \Rightarrow \quad \lambda = \pm i \sqrt{\frac{g}{L}} \quad (14)$$

These are **pure imaginary** numbers, indicating:

- **Oscillatory behavior** with no damping
- **Natural frequency** $\omega_n = \sqrt{g/L}$
- **Neutrally stable** equilibrium (neither attracting nor repelling)

These eigenvalues **exactly characterize** the local behavior of the nonlinear system near equilibrium. For small perturbations, the linear and nonlinear trajectories are indistinguishable to first order.

2.4.6 Where Nonlinearity Emerges

The nonlinearity of the full pendulum—the amplitude-dependent period, the possibility of full rotations—emerges only as perturbations grow large enough that higher-order terms in the Taylor expansion of $\sin \theta$ become significant:

$$\sin \theta = \theta - \frac{\theta^3}{6} + \frac{\theta^5}{120} - \dots \quad (15)$$

Near equilibrium ($\theta \approx 0$), the θ^3 and higher terms are negligible. The tangent space dynamics reign.

2.5 Exact Superposition in the Tangent Space

2.5.1 Superposition Properties

Because the tangent space is a linear vector space, it inherits all properties of linear algebra—including the **superposition principle**.

A linear map satisfies two fundamental properties:

1. **Additivity:** $B(\delta u_1 + \delta u_2) = B\delta u_1 + B\delta u_2$
2. **Homogeneity:** $B(\alpha \delta u) = \alpha B\delta u$ for scalar α

! Exact Superposition Theorem (Fixed State & Fixed Mode)

Superposition holds **exactly** for the **instantaneous** mapping to the tangent space at a fixed state (x, mode) . If we apply two infinitesimal input perturbations δu_1 and δu_2 simultaneously, the resulting state perturbation is exactly the sum of the perturbations that would result from applying each input separately:

$$\delta \dot{x}(\delta u_1 + \delta u_2) = \delta \dot{x}(\delta u_1) + \delta \dot{x}(\delta u_2)$$

The derivative is additive in the perturbations. Its matrix can still couple coordinates; additivity does not imply mechanical independence. The tangent space **moves with the state**, so trajectories are not additive.

This is not an approximation we impose for convenience. It is a **mathematical theorem**, inherited directly from the properties of matrix multiplication and the definition of the derivative.

! Instantaneous Force–Acceleration Superposition (Fixed State & Fixed Mode)

For a mechanical system with equations $M(q)\ddot{q} + h(q, \dot{q}) = \tau + J^\top f$, at a fixed state $(q, \dot{q})$ and fixed contact/constraint mode m , the acceleration is:

$$\ddot{q} = \Phi(q, \dot{q}, m)\tau + \Psi(q, \dot{q}, m)$$

which is affine/linear in applied torques τ and generalized forces f . Therefore, instantaneous acceleration contributions from separate force/torque components **add exactly**. **However**, mode switching (contact/friction transitions), actuator saturation, and inequality constraints (unilateral contacts) make the mapping only **piecewise** linear and can break global superposition. The claim is valid for fixed $(q, \dot{q}, m)$ —not universally across mode boundaries.

2.5.2 Counterfactual Decomposition

At a fixed time, suppose we apply multiple input perturbations $\delta u_1, \delta u_2, \dots, \delta u_k$. We can meaningfully ask: how does each input contribute to the resulting state change?

In the tangent space, the answer is unambiguous. Each input perturbation δu_i produces a corresponding state rate change:

$$\delta \dot{x}_i = B \delta u_i$$

And the total state rate change is simply the sum:

$$\delta \dot{x} = \sum_i \delta \dot{x}_i = B \sum_i \delta u_i \quad (16)$$

This **counterfactual decomposition** allows us to attribute portions of the rate of change to individual inputs. The differential contributions add within the specified coordinates and nominal model. This does not make actuators mechanically independent or identify physiological causes from measured motion.

i Validity Domain

Superposition holds for the **instantaneous** mapping to the tangent space at fixed (x, mode) ; the tangent space **moves with the state**, so trajectories are not additive. When we integrate over finite time intervals, deviations from additivity appear. Finite input changes can move the system through different states and therefore change its derivatives. Such nonlinear interactions occur even in flat Euclidean state space. Intrinsic manifold curvature is a different geometric object and is not required for a nonzero residual.

3 Time Evolution and Moving Tangent Spaces

3.1 Why Global Superposition Fails

If instantaneous superposition holds exactly in each tangent space, why do we say nonlinear systems violate superposition? The answer lies in **time evolution**: the tangent space moves with the state.

The key observation: as the system evolves along a trajectory, the operating point changes—and with it, the tangent space. The Jacobian matrices are not constants; they are functions of the current state:

$$A(t) = \frac{\partial f}{\partial x}(x(t), u(t)), \quad B(t) = \frac{\partial f}{\partial u}(x(t), u(t)) \quad (17)$$

At each instant, the tangent space provides an exact linear description of the local dynamics. But **the tangent space itself moves** as the system evolves.

3.1.1 Geometric Interpretation

Return to the mountain hiker analogy:

- At each point, the ground is locally flat (tangent plane)
- The tangent plane rotates and tilts as the hiker moves
- The mountain is curved even though every tangent plane is flat

For nonlinear dynamics:

- State space is a manifold (possibly curved, complex topology)
- At each point there is a tangent space (exactly linear)
- Trajectories pass through a continuous family of tangent spaces

Global superposition fails not because linearity breaks down at any particular instant, but because the linear structure itself evolves. Two solutions we might wish to superpose live in different tangent spaces at each moment in time. Adding them is geometrically meaningless—like trying to add vectors from different vector spaces.

3.1.2 Nonlinearity as Tangent Space Variation

This perspective transforms how we think about nonlinearity. The “nonlinear effects” that distinguish, say, a large-amplitude pendulum swing from a small one are not violations of linearity per se. They are consequences of:

1. The trajectory passing through regions where tangent spaces differ significantly
2. The accumulated effect of these differences over time
3. The state and input dependence of the vector-field derivatives, which is not a measurement of intrinsic manifold curvature

Nonlinearity is **relocated** from the local structure (which is exactly linear) to the global geometry (how local structures fit together).

3.2 Exactness in the Limit

3.2.1 Taylor Expansion Analysis

Consider a finite timestep Δt . The state at time $t + \Delta t$ can be expressed via Taylor expansion:

$$x(t + \Delta t) = x(t) + f(x(t), u(t)) \Delta t + \frac{1}{2} \frac{d}{dt} f(x(t), u(t)) \Delta t^2 + O(\Delta t^3) \quad (18)$$

The terms break down as:

1. **Zeroth order:** $x(t)$ — current state
2. **First order:** $f(x(t), u(t)) \Delta t$ — tangent space prediction (linear contribution)
3. **Second order and higher:** $O(\Delta t^2)$ — curvature effects

3.2.2 The Limit Argument

As $\Delta t \rightarrow 0$, the higher-order terms become negligible compared to the first-order term:

$$\lim_{\Delta t \rightarrow 0} \frac{\|x(t + \Delta t) - x(t) - f(x(t), u(t)) \Delta t\|}{\Delta t} = 0 \quad (19)$$

This is the precise sense in which **linearization is exact in the limit**. The first-order evolution—governed entirely by the tangent space dynamics—captures the behavior of the nonlinear system up to terms that become arbitrarily small relative to the timestep.

! Infinitesimal Exactness

Linearization is exact in the limit $\Delta t \rightarrow 0$. This is not a weaker form of exactness than we find elsewhere in mathematics—it is the same sense in which derivatives, integrals, and differential equations themselves are exact.

The derivative $\dot{x} = f(x, u)$ is defined as a limit; it tells us the instantaneous rate of change, not the change over any finite interval. Linearization inherits this same infinitesimal exactness.

The tangent space is not an approximation to the nonlinear system—it is the infinitesimal structure of the system, captured exactly.

3.2.3 Variational Equation

For a perturbation $\delta x(t)$ from a nominal trajectory $x(t)$, the evolution is governed by the **variational equation**:

$$\frac{d}{dt}\delta x = A(t)\delta x + B(t)\delta u \quad (20)$$

This is a **linear time-varying (LTV) system**. At each instant, it's exactly linear. The time-variation encodes how the tangent space changes along the trajectory.

4 Local Optimal Control: LQR and Lyapunov Stability

4.1 Connection to Linear Quadratic Regulation

The theoretical framework developed thus far is not merely academic—it underlies some of the most successful techniques in control engineering.

4.1.1 The LQR Problem

Linear Quadratic Regulation (LQR) solves the following problem:

Given:

- Linear system: $\dot{x} = Ax + Bu$
- Quadratic cost: $J = \int_0^\infty (x^T Q x + u^T R u) dt$
- Matrices $Q \succeq 0$ (positive semidefinite), $R \succ 0$ (positive definite)

Find: Optimal feedback control law minimizing J

Solution: Linear state feedback $u = -Kx$ where gain K is computed by solving the **algebraic Riccati equation**:

$$A^T P + PA - PBR^{-1}B^T P + Q = 0 \quad (21)$$

with $K = R^{-1}B^T P$. Standard sufficient assumptions for the stabilizing infinite-horizon solution are stabilizability of (A, B) and detectability of $(Q^{1/2}, A)$. Select the stabilizing positive-semidefinite solution; the matrix equation alone does not guarantee that an arbitrary root defines an optimal stabilizer.

4.1.2 What a Nonlinear LQR Design Establishes

At an equilibrium, a differentiable nonlinear system has a linear first variation. If the resulting closed-loop Jacobian is Hurwitz, the smooth nonlinear closed loop is locally asymptotically stable. This is a local result with a model-dependent neighborhood. It does not establish that the linear feedback globally minimizes the nonlinear objective.

A C^2 cost has a second-order Taylor model. Its linear term vanishes at an unconstrained stationary minimum, but the Hessian may be singular or zero: $\ell(x) = x^4$ has no nonzero quadratic leading term. Along a general nominal trajectory, retain cost gradients and state-input cross derivatives rather than assuming each stage is at a minimum.

Along a dynamically feasible moving reference, time-varying LQR propagates a value approximation backward from the terminal cost. Solving independent equilibrium Riccati equations

at each instant is a different design. The finite-horizon recursion, nonlinear rollout, input constraints, and stability argument must be checked separately. See the [reviewed optimal-control chapter](#).

4.1.3 Gain Scheduling Requires Its Own Check

Interpolating locally stabilizing gains does not automatically stabilize the scheduled nonlinear system. For a scheduled Lyapunov metric $P(\rho)$, its derivative contains $\sum_i P_{\rho_i} \dot{\rho}_i$ as well as the closed-loop Jacobian terms. A common Lyapunov function, appropriate scheduling-rate bounds, or a direct trajectory analysis can supply a certificate. Frozen-point eigenvalues alone do not.

4.2 Lyapunov Stability Analysis

4.2.1 Lyapunov's Indirect Method

One of the most powerful tools for analyzing nonlinear stability is **Lyapunov's indirect method**:

Theorem: Lyapunov's Indirect Method

Consider $\dot{x} = f(x)$ with equilibrium x_e (i.e., $f(x_e) = 0$). Let $A = \frac{\partial f}{\partial x}(x_e)$ be the Jacobian at equilibrium.

1. If all eigenvalues of A have strictly negative real parts ($\text{Re}(\lambda_i) < 0$), then x_e is **locally asymptotically stable**
2. If any eigenvalue of A has strictly positive real part ($\text{Re}(\lambda_j) > 0$), then x_e is **unstable**
3. If all eigenvalues have non-positive real parts with some on imaginary axis, stability is **inconclusive** (higher-order analysis required)

4.2.2 Why This Works

This is a remarkable result: we draw conclusions about a **nonlinear system**—which may exhibit chaos, limit cycles, or other complex behaviors—based solely on a **linear analysis**.

The justification is precisely the exactness we have established:

1. Near equilibrium, the tangent space dynamics dominate
2. The linearization $\delta\dot{x} = A\delta x$ captures the essential character of nearby trajectories
3. Eigenvalues of A determine whether perturbations grow or decay

4. For small perturbations, this linear analysis is exact to leading order

i The Pendulum Example Revisited

Recall the pendulum at $x_e = [0, 0]^T$ has:

$$A = \begin{bmatrix} 0 & 1 \\ -g/L & 0 \end{bmatrix}, \quad \lambda = \pm i\sqrt{g/L}$$

The eigenvalues are pure imaginary (on the borderline), so Lyapunov's indirect method is **inconclusive**. Indeed, the pendulum equilibrium is:

- Stable (nearby trajectories stay nearby)
- But not asymptotically stable (they don't converge to equilibrium)
- Neutrally stable—consistent with conservative mechanical system

For the inverted pendulum at $x_e = [\pi, 0]^T$, one eigenvalue would be positive, confirming instability.

4.2.3 Connection to Tangent Space Framework

Lyapunov stability analysis via linearization succeeds because:

- The tangent space at equilibrium captures exact local dynamics
- Eigenvalues encode local growth/decay rates exactly
- For stability, only local behavior matters (by definition)

This yields rigorous local conclusions under the stated smoothness and spectral assumptions. Global attraction requires an additional argument over a specified domain.

4.3 Practical Implications

4.3.1 Design Philosophy

The tangent space perspective suggests a design philosophy:

1. **Locally optimal:** Design controllers that are optimal in each tangent space
2. **Globally aware:** Account for how tangent spaces vary (gain scheduling, adaptive control)
3. **Residual monitoring:** Track when nonlinear effects (curvature) become significant

4.3.2 When Linear Techniques Suffice

Linear control techniques (LQR, pole placement, loop shaping) work well when:

- Operating region is small (tangent spaces don't vary much)
- Trajectory stays near equilibrium (single tangent space dominates)
- Curvature is mild (residuals small)

4.3.3 When Nonlinear Techniques Required

Nonlinear analysis or controller designs may be useful in the following situations; these features alone do not prove that a linear controller must fail:

- Large operating regions (tangent spaces vary significantly)
- Complex manifold topology (multiple equilibria, limit cycles)
- Strong curvature (large residuals)

5 Residuals, Reference Motion, and Nonlinearity

5.1 An Interaction Requires Four Comparable Trajectories

Fix an initial state, model, time interval, and nominal input $\bar{u}$. Let x_{00} use $\bar{u}$, x_{10} use $\bar{u} + \epsilon v_1$, x_{01} use $\bar{u} + \epsilon v_2$, and x_{11} use $\bar{u} + \epsilon(v_1 + v_2)$. In a common state coordinate chart, define

$$r(t_1) = x_{11}(t_1) - x_{10}(t_1) - x_{01}(t_1) + x_{00}(t_1). \quad (22)$$

The fourth term is the **evolved nominal trajectory**, not the initial state. With zero nominal input it is the evolved zero-input trajectory. For the scalar linear system $\dot{x} = ax + u$, constant u , and duration T , the endpoint is $e^{aT}x_0 + u(e^{aT} - 1)/a$ for $a \neq 0$. The four-trajectory residual is zero. Replacing its last term by x_0 gives $x_0(1 - e^{aT})$, a false interaction even in this linear system.

For $\dot{x} = u^2$ on the Euclidean line and constant perturbations, the same residual is $2\epsilon^2 v_1 v_2 T$. The state space is flat; the input-to-flow map is nonlinear. Ordinary Hessians depend on coordinates and are not intrinsic Riemann curvature tensors. On a manifold, state subtraction requires a declared chart or retraction and a consistent comparison point.

5.2 First Variations and the Mixed Second Variation

For a fixed finite horizon, assume $f(x, u, t)$ is C^2 in state and input on a neighborhood containing the relevant trajectories, with sufficient time regularity and bounded input directions. Let

$$\dot{\xi}_i = A(t)\xi_i + B(t)v_i, \quad \xi_i(t_0) = 0, \quad A = f_x(\bar{x}, \bar{u}, t), \quad B = f_u(\bar{x}, \bar{u}, t).$$

These are exact first variations along one nominal trajectory. Their sum obeys the same linear equation. Define η_{12} as the mixed second variation of the solution with respect to the two input amplitudes. It satisfies

$$\dot{\eta}_{12} = A\eta_{12} + S_{12}, \quad \eta_{12}(t_0) = 0,$$

$$S_{12} = f_{xx}[\xi_1, \xi_2] + f_{xu}[\xi_1, v_2] + f_{xu}[\xi_2, v_1] + f_{uu}[v_1, v_2]. \quad (23)$$

Every derivative is evaluated along the same nominal state and input. For a control-affine system, $f_{uu} = 0$, but f_{xu} can be nonzero when the input vector fields depend on state. The interaction is

$$r(t_1) = \epsilon^2 \eta_{12}(t_1) + o(\epsilon^2), \quad \eta_{12}(t_1) = \int_{t_0}^{t_1} \Phi(t_1, t) S_{12}(t) dt. \quad (24)$$

There is no factor of $1/2$ on this mixed interaction: the two symmetric mixed terms in the Taylor expansion cancel that factor. A single trajectory's second-order Taylor error is a different quantity.

5.3 Transport Is Part of the Bound

For a chosen consistent norm and fixed interval,

$$\|r(t_1)\| \leq \epsilon^2 \int_{t_0}^{t_1} \|\Phi(t_1, t)\| \|S_{12}(t)\| dt + o(\epsilon^2). \quad (25)$$

The Jacobian matters through Φ and the first variations inside S_{12} . For $\dot{x} = ax + u^2$, constant directions about $\bar{u} = 0$ give $r = 2\epsilon^2 v_1 v_2 (e^{aT} - 1)/a$. A constant Hessian source can therefore be amplified exponentially. A bound proportional to T is valid only when its coefficient controls this transport and the other terms over the entire stated interval; it is not a universal linear growth law.

5.4 What a Small Residual Does Not Establish

A small interaction residual does not bound every prediction error. For instance, two input directions may have no mutual interaction while each has its own nonlinear self-effect. A zero denominator also makes $\|r\|/\|\delta x\|$ unusable. Choose physical scales or a metric, a reference magnitude bounded away from zero, and a task tolerance:

$$\frac{\|r\|_W}{s_{\text{reference}}} \ll 1, \quad s_{\text{reference}} > 0. \quad (26)$$

This is an interaction diagnostic, not a stability or optimality certificate. Separately check finite-model prediction errors, disturbances, discretization error, and active contact modes. A computation should demonstrate convergence under tighter integration tolerances before calling its residual physical.

Part II: Integral Superposition and Transport

6 Fundamentals of Integral Accumulation

7 Integration Preserves Linearity

7.1 From Infinitesimal to Finite Time

In Part I, we established that in the tangent space at each instant:

$$\delta \dot{x}(t) = A(t)\delta x(t) + B(t)\delta u(t)$$

This is exact superposition at a single moment. The natural question: **what happens when we integrate over time?**

The key insight: **integration is a linear operation.** If pointwise superposition holds at every instant, then integrated superposition holds over intervals.

7.2 The Integration Theorem

! Theorem: Integral Linearity

For variations $\delta x_1(t)$ and $\delta x_2(t)$ satisfying:

$$\delta \dot{x}_i(t) = A(t)\delta x_i(t) + B(t)\delta u_i(t), \quad i = 1, 2$$

their sum $\delta x(t) = \delta x_1(t) + \delta x_2(t)$ satisfies:

$$\delta \dot{x}(t) = A(t)\delta x(t) + B(t)\delta u(t)$$

where $\delta u(t) = \delta u_1(t) + \delta u_2(t)$.

Proof: Follows immediately from linearity of differentiation and matrix multiplication.

□

Integrating both sides from t_0 to t_1 :

$$\int_{t_0}^{t_1} \delta \dot{x}(t) dt = \int_{t_0}^{t_1} [A(t)\delta x(t) + B(t)\delta u(t)] dt \quad (27)$$

The left side is simply:

$$\int_{t_0}^{t_1} \delta \dot{x}(t) dt = \delta x(t_1) - \delta x(t_0) \quad (28)$$

This is the **endpoint variation**—the change in state perturbation from start to finish.

7.3 What Superposes: Variations, Not Causes

Critical Distinction

What superposes is the **variation between endpoints**, not the “fractional cause” of the final state. This is a subtle but crucial point.

Consider two inputs $u_1(t)$ and $u_2(t)$ applied from t_0 to t_1 :

- We can exactly compute: $\Delta x_1 = x_1(t_1) - x(t_0)$ and $\Delta x_2 = x_2(t_1) - x(t_0)$
- If both are applied: $\Delta x_{12} = x_{12}(t_1) - x(t_0)$
- The integrated variation equation tells us how Δx_1 and Δx_2 relate to Δx_{12}

But we cannot meaningfully say “input 1 caused 40% of the final state”—causation is not additive in nonlinear systems. Variation is additive (to leading order).

8 State Transition Operator

8.1 Solution of the Variational Equation

The variational equation:

$$\frac{d}{dt} \delta x = A(t) \delta x + B(t) \delta u \quad (29)$$

is a linear time-varying (LTV) system. Its solution has the form:

$$\delta x(t_1) = \Phi(t_1, t_0) \delta x(t_0) + \int_{t_0}^{t_1} \Phi(t_1, \tau) B(\tau) \delta u(\tau) d\tau \quad (30)$$

where $\Phi(t_1, t_0)$ is the **state transition operator** (or fundamental matrix solution).

8.2 Properties of Φ

The state transition operator satisfies:

1. **Initial condition:** $\Phi(t, t) = I$ (identity matrix)
2. **Semigroup property:** $\Phi(t_2, t_0) = \Phi(t_2, t_1)\Phi(t_1, t_0)$
3. **Evolution equation:** $\frac{\partial}{\partial t_1}\Phi(t_1, t_0) = A(t_1)\Phi(t_1, t_0)$
4. **Inverse:** $\Phi(t_0, t_1) = \Phi(t_1, t_0)^{-1}$

These properties make Φ a linear map that “transports” perturbations from t_0 to t_1 while accounting for how the tangent space evolves.

8.3 Interpretation: Transport Then Accumulate

Equation Equation 30 has two terms:

1. $\Phi(t_1, t_0)\delta x(t_0)$: Initial perturbation transported to t_1
2. $\int_{t_0}^{t_1} \Phi(t_1, \tau)B(\tau)\delta u(\tau)d\tau$: Accumulated effect of inputs, transported from application time τ to final time t_1

The accumulated input effect is a **linear integral**—each infinitesimal contribution $B(\tau)\delta u(\tau)d\tau$ is transported to t_1 via $\Phi(t_1, \tau)$, and the results superpose exactly.

! Integral Superposition Principle

The state transition operator formulation shows that **integrated variations superpose exactly through linear transport**. The nonlinearity of the original system is encoded in how Φ varies with the nominal trajectory, not in any failure of linearity within the variational dynamics.

9 Time-Invariant Special Case

9.1 Constant Jacobian

If the system linearization is time-invariant (A and B constant), the state transition operator simplifies:

$$\Phi(t_1, t_0) = e^{A(t_1 - t_0)} \quad (31)$$

The matrix exponential is defined via:

$$e^{At} = I + At + \frac{(At)^2}{2!} + \frac{(At)^3}{3!} + \dots$$

and can be computed via eigendecomposition if A is diagonalizable.

9.2 Linear Time-Invariant (LTI) Systems

For LTI systems, the solution becomes:

$$\delta x(t_1) = e^{A(t_1-t_0)}\delta x(t_0) + \int_{t_0}^{t_1} e^{A(t_1-\tau)}B\delta u(\tau)d\tau \quad (32)$$

This is the classical result from linear systems theory. The integral term shows explicitly how contributions from different times superpose—each input at time τ contributes $e^{A(t_1-\tau)}B\delta u(\tau)d\tau$ to the final state, and these contributions add linearly.

10 Variational Dynamics and Sensitivity Analysis

11 Forward Sensitivity

11.1 Sensitivity to Initial Conditions

The partial derivative:

$$S_x(t) = \frac{\partial x(t)}{\partial x(t_0)} \quad (33)$$

describes how the state at time t depends on the initial condition. From the state transition operator:

$$S_x(t) = \Phi(t, t_0) \quad (34)$$

This matrix evolves according to:

$$\frac{d}{dt} S_x(t) = A(t) S_x(t), \quad S_x(t_0) = I \quad (35)$$

11.2 Sensitivity to Inputs

Similarly, sensitivity to an input applied at time τ :

$$S_u(t, \tau) = \frac{\partial x(t)}{\partial u(\tau)} = \Phi(t, \tau) B(\tau) \quad (36)$$

This tells us: if we perturb the input at time τ by $\delta u(\tau)$, the state at time $t > \tau$ changes by approximately $\Phi(t, \tau) B(\tau) \delta u(\tau)$.

11.3 Integrated Sensitivity

The total sensitivity to an input trajectory $u(\cdot)$ over $[t_0, t_1]$ is:

$$\frac{\partial x(t_1)}{\partial u(\cdot)} = \int_{t_0}^{t_1} \Phi(t_1, \tau) B(\tau) d\tau \quad (37)$$

This is a **linear functional** mapping input functions to state changes—the embodiment of integral superposition.

12 Adjoint (Costate) Dynamics

12.1 Backward Sensitivity

Forward sensitivity tells us how the final state depends on inputs. **Adjoint dynamics** answer the dual question: how does a scalar cost depend on the state trajectory?

Consider a cost functional:

$$J = \phi(x(t_1)) + \int_{t_0}^{t_1} L(x(t), u(t), t) dt \quad (38)$$

where:

- $\phi(x(t_1))$: terminal cost
- $L(x, u, t)$: running cost (Lagrangian)

Define the **costate** (adjoint variable):

$$\lambda(t) = \frac{\partial J}{\partial x(t)} \quad (39)$$

This is the sensitivity of the cost to state perturbations at time t .

12.2 Adjoint Evolution Equation

The costate evolves backward in time:

$$\frac{d}{dt}\lambda(t) = -A(t)^T\lambda(t) - \frac{\partial L}{\partial x}(x(t), u(t), t) \quad (40)$$

with terminal condition:

$$\lambda(t_1) = \frac{\partial \phi}{\partial x}(x(t_1)) \quad (41)$$

This is the **adjoint equation**—it integrates sensitivities backward from the final time.

12.3 Physical Interpretation

The costate $\lambda(t)$ represents the “shadow price” or “marginal value” of the state at time t :

- If we perturb $x(t)$ by $\delta x(t)$, the cost changes by approximately $\lambda(t)^T \delta x(t)$
- Integrating backward accumulates these marginal values
- At t_0 , $\lambda(t_0)^T \delta x(t_0)$ gives the total cost impact of initial condition perturbations

12.4 Duality: Forward and Backward

- **Forward sensitivity** (Φ): How inputs propagate to states
- **Backward sensitivity** (λ): How states contribute to costs

These are dual perspectives on the same system, connected through:

$$\frac{dJ}{du(t)} = B(t)^T \lambda(t) + \frac{\partial L}{\partial u}(x(t), u(t), t) \quad (42)$$

This is the basis for gradient-based optimization in optimal control.

13 Physical Interpretations: Impulse and Work

14 Force-Momentum: Impulse Integral

14.1 Newton's Second Law

For a mechanical Lagrangian $L_m = T - V$ with $p = \partial L_m / \partial \dot{q} = M(q)\dot{q}$, the Euler–Lagrange momentum equation can be written:

$$\frac{dp}{dt} = f(q, \dot{q}) + u \quad (43)$$

Here u is the chosen generalized applied load and $f = \partial L_m / \partial q + Q_{\text{other}}$. The derivative $\partial L_m / \partial q$ includes configuration-dependent kinetic terms as well as potential forces. Canonical generalized momentum need not equal a Cartesian momentum component. For Cartesian translation with constant mass, the equation reduces to Newton's law with all other external forces included in f .

14.2 Impulse Definition

The **impulse** delivered by a force u over $[t_0, t_1]$ is:

$$J = \int_{t_0}^{t_1} u(\tau) d\tau \quad (44)$$

From Newton's second law:

$$\int_{t_0}^{t_1} \frac{dp}{dt} dt = p(t_1) - p(t_0) = \int_{t_0}^{t_1} [f + u] dt \quad (45)$$

If the system starts at rest ($\dot{q}(t_0) = 0$) and the integral of every retained term in f is demonstrably negligible on the stated interval, then approximately:

$$p(t_1) \approx \int_{t_0}^{t_1} u(\tau) d\tau = J \quad (46)$$

Impulse equals change in momentum—a direct consequence of integrating Newton's law.

14.3 Superposition of Impulses

If two forces u_1 and u_2 are applied:

$$J_{12} = \int_{t_0}^{t_1} [u_1(\tau) + u_2(\tau)] d\tau = \int_{t_0}^{t_1} u_1(\tau) d\tau + \int_{t_0}^{t_1} u_2(\tau) d\tau = J_1 + J_2 \quad (47)$$

This is **exact**—impulses superpose because integration is linear. The resulting momentum change is:

$$p(t_1) = p(t_0) + J_1 + J_2 + \int_{t_0}^{t_1} f(q(t), \dot{q}(t)) dt \quad (48)$$

This is exact bookkeeping on the combined trajectory. The remaining integral is not generally higher order or small; gravity can contribute at first order in elapsed time. If force laws or feedback depend on the state, their actual histories also change across separate counterfactual runs.

i Interpretation

The integral nature of impulse guarantees linear accumulation of force effects. Each infinitesimal force contribution $u(\tau)d\tau$ adds linearly to the total momentum change. This is integral superposition applied to mechanical systems.

15 Power-Energy: Work Integral

15.1 Power and Work

For a mechanical system, **power** delivered by a force is:

$$P(t) = u(t)^T \dot{q}(t) \quad (49)$$

The **work** done over $[t_0, t_1]$ is:

$$W = \int_{t_0}^{t_1} P(\tau) d\tau = \int_{t_0}^{t_1} u(\tau)^T \dot{q}(\tau) d\tau \quad (50)$$

For generalized load u paired with $\dot{q}$, this is the work of that load. The kinetic-energy balance must include the work W_{other} of all other forces, including gravity when it is not absorbed into potential energy:

$$W + W_{\text{other}} = E_k(t_1) - E_k(t_0) \quad (51)$$

Here $E_k = \frac{1}{2}\dot{q}^T M \dot{q}$. For a time-independent conservative potential and only dissipative loads $-d$ besides u , the mechanical-energy balance is $\Delta(E_k + V) = \int (u - d)^T \dot{q} dt$. Ideal stationary constraint reactions do no work; moving supports can. These assumptions must be stated before equating actuator work with an energy change.

15.2 Superposition of Work

If two forces u_1 and u_2 are applied:

$$W_{12} = \int_{t_0}^{t_1} [u_1 + u_2]^T \dot{q} d\tau = \int_{t_0}^{t_1} u_1^T \dot{q} d\tau + \int_{t_0}^{t_1} u_2^T \dot{q} d\tau = W_1 + W_2 \quad (52)$$

Again, **exact** additivity due to integral linearity.

However, there's a subtlety: $\dot{q}(\tau)$ itself depends on the forces. So while the integral is linear in the integrand, the velocity trajectory is nonlinear in the forces. The work contributions are additive, but their relationship to the final energy is not simple due to cross-coupling.

Variation vs. Causation

The force-work terms add along the same combined trajectory, providing a valid signed energy budget when all terms are included. Those terms are not the works obtained by running each input alone, and normalized work fractions do not uniquely identify physiological causation or efficiency. Negative work and an initial energy store can also make naive percentage interpretations misleading.

This reinforces: **integrated variations** superpose; **fractional causation** does not.

16 General Principle: Integral Accumulation

Both impulse and work illustrate a general principle:

! Integral Accumulation Principle

Along a fixed trajectory $x(t)$, for any quantity defined as an integral of a linear function of inputs:

$$\Psi = \int_{t_0}^{t_1} g(x(t), u(t), t) dt$$

where g is linear in u with $x(t)$ held fixed, the contributions from different inputs superpose exactly:

$$\Psi(u_1 + u_2) = \Psi(u_1) + \Psi(u_2)$$

Important caveat: This integral identity holds for any such input histories evaluated along the same fixed trajectory $x(t)$; the histories need not be infinitesimal. If $x(t)$ itself depends nonlinearly on u (as it generally does in closed-loop or trajectory optimization settings), then the principle does not hold globally — changing u changes the trajectory, and the two contributions cannot be evaluated independently.

Impulse and work are special cases where $g = u$ and $g = u^T \dot{q}$ respectively. The linearity of integration ensures exact additivity of these quantities, even as the underlying dynamics remain nonlinear.

17 Finite-Horizon Residuals and Error Quantification

17.1 Keep the Baseline and the Perturbation Definition Fixed

The four-trajectory interaction in Equation 22 can also be written using finite deviations $\Delta x_{ij} = x_{ij} - x_{00}$:

$$r(t_1) = x_{11}(t_1) - x_{10}(t_1) - x_{01}(t_1) + x_{00}(t_1). \quad (53)$$

$$r = \Delta x_{11} - \Delta x_{10} - \Delta x_{01}. \quad (54)$$

These finite deviations differ from the first variations ξ_i , whose equation is linear and whose interaction is exactly zero. The notation must not quietly turn one into the other.

For a smooth solution map on a fixed interval,

$$\|r\| = O(\epsilon^2). \quad (55)$$

If $\|\Phi(t_1, t)\| \|S_{12}(t)\| \leq C_T$ throughout an interval of duration T , then

$$\|r\| \leq \epsilon^2 C_T T + o(\epsilon^2). \quad (56)$$

C_T can grow rapidly with the horizon. Quadratic order does not mean exact quadrupling at arbitrary amplitudes: the mixed coefficient can vanish, higher orders can matter, and contacts can destroy smooth dependence. Convergence in perturbation amplitude should be tested at a fixed horizon.

17.2 Interaction and Single-Trajectory Approximation Error

The relevant Hessians are

$$H_x = f_{xx}, \quad H_u = f_{uu}, \quad H_{xu} = f_{xu}. \quad (57)$$

The mixed interaction uses S_{12} from Equation 23:

$$r(t_1) = \epsilon^2 \int_{t_0}^{t_1} \Phi(t_1, t) S_{12}(t) dt + o(\epsilon^2). \quad (58)$$

By comparison, a single-input expansion $x_\epsilon = \bar{x} + \epsilon\xi + \frac{1}{2}\epsilon^2\eta + o(\epsilon^2)$ has a self-variation source $f_{xx}[\xi, \xi] + 2f_{xu}[\xi, v] + f_{uu}[v, v]$. Both calculations need transport. Neither identifies intrinsic manifold curvature from a measured residual.

17.3 A Task-Based Interpretation for the Swing

Specify whether comparisons use the same elapsed time or an event such as first ball contact. A perturbed trajectory may reach that event at a different time, so a contact-event endpoint map must include event-time sensitivity; smooth-flow expressions alone do not provide it. A changed contact mode requires a hybrid analysis.

In a common coordinate chart, compare residuals using declared scales:

$$\chi = \|r\|_W / s_{\text{reference}}, \quad s_{\text{reference}} > 0. \quad (59)$$

If the chosen perturbation response scale is proportional to ϵ , a bounded mixed response gives a diagnostic of order

$$\chi = O(\epsilon C_T T). \quad (60)$$

For a prescribed tolerance $\chi_{\max}$, a leading-order estimate is

$$\epsilon C_T T \lesssim \chi_{\max}, \quad (61)$$

with scales absorbed into C_T . It is meaningful only where the derivative bounds and remainder estimates hold. There is no universal threshold such as 0.1 at which a nonlinear controller becomes necessary.

Applied to a swing model, this procedure can quantify how two chosen input interventions interact in club position, face orientation, or speed. It does not recover a unique causal allocation of observed performance to muscles. Geometry, state-dependent leverage, activation, shaft storage, feedback, and impact timing can all alter the endpoint map. Compare those mechanisms through explicit model variants and measured uncertainties.

Part III: Hamiltonian Structure and Optimal Control

 Work in Progress

Part III is a draft/incomplete section. The following provides the conceptual framework and key equations but requires substantial expansion — detailed derivations, complete algorithms, and worked examples are not yet present. Treat this section as a preliminary outline, not a finished exposition.

18 Variational Principles and Hamiltonian Formulation

18.1 Action Functional and Euler-Lagrange Equations

18.1.1 Lagrangian Mechanics

For conservative mechanics, Hamilton's stationary-action principle varies configurations $q(t)$ with fixed endpoints in $S = \int L(q, \dot{q}, t) dt$; stationarity need not be a minimum. External generalized forces enter the Lagrange–d'Alembert formulation. Optimal control is a distinct constrained variational problem. For its running objective L (which need not be kinetic minus potential energy), define:

$$S[x(\cdot), u(\cdot)] = \int_{t_0}^{t_1} L(x(t), u(t), t) dt \quad (62)$$

Here L denotes an optimization running cost. Using a different symbol such as ℓ avoids confusing it with the mechanical Lagrangian.

A candidate minimizing trajectory satisfies necessary conditions for the specified dynamics, endpoint conditions, and admissible inputs. A physical trajectory does not generally minimize an arbitrary chosen control objective.

18.1.2 Hamiltonian Formulation

Introduce the **Hamiltonian**:

$$H(x, u, \lambda, t) = L(x, u, t) + \lambda^T f(x, u) \quad (63)$$

where λ is the **costate** (Lagrange multiplier enforcing dynamics).

The optimal trajectory satisfies:

$$\begin{aligned}
\dot{x} &= \frac{\partial H}{\partial \lambda} = f(x, u) && \text{(state equation)} \\
\dot{\lambda} &= -\frac{\partial H}{\partial x} && \text{(costate equation)} \\
0 &= \frac{\partial H}{\partial u} && \text{(optimality condition)}
\end{aligned} \tag{64}$$

These are the normal, unconstrained interior stationarity conditions for a minimization convention. With bounded inputs, minimize this Hamiltonian over the admissible set instead of universally imposing $H_u = 0$. Terminal cost and endpoint constraints determine costate transversality; stationarity alone does not prove a minimum.

i Connection to Previous Content

The costate equation $\dot{\lambda} = -\frac{\partial H}{\partial x}$ is the adjoint equation from Chapter 6. The Hamiltonian framework unifies forward dynamics ($\dot{x}$), backward sensitivity ($\dot{\lambda}$), and optimality ($\frac{\partial H}{\partial u} = 0$) into a single variational principle.

18.2 Costate as Sensitivity

18.2.1 Physical Meaning

The costate $\lambda(t)$ represents:

1. **Cost sensitivity:** $\lambda(t)^T \delta x(t) \approx \delta J$ (perturbation in cost)
2. **Shadow price:** Marginal value of the state at time t
3. **Momentum** in some mechanical interpretations

18.2.2 Backward Integration

The costate evolves backward in time from the terminal condition:

$$\lambda(t_1) = \frac{\partial \phi}{\partial x}(x(t_1)) \tag{65}$$

Integrating backward accumulates the cost implications of state perturbations along the trajectory.

! Duality Perspective

- **Forward problem:** Given $x(t_0)$ and $u(\cdot)$, integrate $\dot{x} = f$ forward to find $x(t_1)$
- **Backward problem:** Given $x(t_1)$ and cost structure, integrate $\dot{\lambda} = -\frac{\partial H}{\partial x}$ backward to find $\lambda(t_0)$

These are dual formulations of the same optimal control problem.

18.3 Detailed Variational Derivation

18.3.1 Calculus of Variations

Consider an optimal control problem:

$$\min_{u(\cdot)} J = \phi(x(t_1), t_1) + \int_{t_0}^{t_1} L(x(t), u(t), t) dt \quad (66)$$

subject to:

$$\dot{x}(t) = f(x(t), u(t)), \quad x(t_0) = x_0 \text{ given} \quad (67)$$

where ϕ is the terminal cost and L is the running cost (Lagrangian).

18.3.2 Augmented Functional

Enforce the dynamics constraint using Lagrange multipliers $\lambda(t)$ (the costate):

$$\tilde{J} = \phi(x(t_1), t_1) + \int_{t_0}^{t_1} [L(x, u, t) + \lambda^T(f(x, u) - \dot{x})] dt \quad (68)$$

Define the **Hamiltonian**:

$$H(x, u, \lambda, t) = L(x, u, t) + \lambda^T f(x, u) \quad (69)$$

Then:

$$\tilde{J} = \phi(x(t_1), t_1) + \int_{t_0}^{t_1} [H(x, u, \lambda, t) - \lambda^T \dot{x}] dt \quad (70)$$

18.3.3 First Variation

For optimality, the first variation $\delta\tilde{J} = 0$ for all admissible perturbations δx , δu , $\delta\lambda$.

Computing $\delta\tilde{J}$:

$$\delta\tilde{J} = \frac{\partial\phi}{\partial x}(t_1)\delta x(t_1) + \int_{t_0}^{t_1} \left[\frac{\partial H}{\partial x}\delta x + \frac{\partial H}{\partial u}\delta u + \frac{\partial H}{\partial \lambda}\delta\lambda - \lambda^T \delta\dot{x} \right] dt \quad (71)$$

Integrate the $\lambda^T \delta\dot{x}$ term by parts:

$$- \int_{t_0}^{t_1} \lambda^T \delta\dot{x} dt = -\lambda(t_1)^T \delta x(t_1) + \lambda(t_0)^T \delta x(t_0) + \int_{t_0}^{t_1} \dot{\lambda}^T \delta x dt \quad (72)$$

Since $x(t_0)$ is fixed, $\delta x(t_0) = 0$. Combining:

$$\delta\tilde{J} = \left[\frac{\partial\phi}{\partial x}(t_1) - \lambda(t_1) \right]^T \delta x(t_1) + \int_{t_0}^{t_1} \left[\frac{\partial H}{\partial x} + \dot{\lambda} \right]^T \delta x dt + \int_{t_0}^{t_1} \frac{\partial H}{\partial u} \delta u dt + \int_{t_0}^{t_1} \frac{\partial H}{\partial \lambda} \delta\lambda dt \quad (73)$$

18.3.4 Optimality Conditions

For $\delta\tilde{J} = 0$ for all variations:

1. **Costate boundary condition:** $\lambda(t_1) = \frac{\partial\phi}{\partial x}(x(t_1), t_1)$
2. **Costate equation:** $\dot{\lambda} = -\frac{\partial H}{\partial x}$
3. **Optimality condition:** $\frac{\partial H}{\partial u} = 0$ (unconstrained case)
4. **State equation:** $\dot{x} = \frac{\partial H}{\partial \lambda} = f(x, u)$

These are **Pontryagin's necessary conditions** for optimality.

! Pontryagin's Maximum Principle

For problems with control constraints $u \in \mathcal{U}$, the optimality condition becomes:

$$u^*(t) = \arg \max_{u \in \mathcal{U}} H(x^*(t), u, \lambda^*(t), t) \quad (74)$$

The optimal control maximizes (or minimizes, depending on convention) the Hamiltonian pointwise in time.

18.4 Examples and Applications

18.4.1 Example 1: Minimum-Energy Problem

Problem: Minimize energy expenditure while transferring between states.

$$\min J = \frac{1}{2} \int_{t_0}^{t_1} u^T R u \, dt \quad (75)$$

subject to linear dynamics $\dot{x} = Ax + Bu$, with fixed endpoints $x(t_0) = x_0$, $x(t_1) = x_1$.

Hamiltonian:

$$H = \frac{1}{2} u^T R u + \lambda^T (Ax + Bu) \quad (76)$$

Optimality condition ($\frac{\partial H}{\partial u} = 0$):

$$Ru + B^T \lambda = 0 \quad \Rightarrow \quad u^* = -R^{-1} B^T \lambda \quad (77)$$

Costate equation:

$$\dot{\lambda} = -A^T \lambda \quad (78)$$

Solution: Solve the two-point boundary value problem (TPBVP):

$$\begin{bmatrix} \dot{x} \\ \dot{\lambda} \end{bmatrix} = \begin{bmatrix} A & -BR^{-1}B^T \\ 0 & -A^T \end{bmatrix} \begin{bmatrix} x \\ \lambda \end{bmatrix} \quad (79)$$

with $x(t_0) = x_0$, $x(t_1) = x_1$.

This can be solved analytically via matrix exponentials or numerically via shooting/collocation methods.

18.4.2 Example 2: Time-Optimal Control

Problem: Minimize transfer time with bounded control $|u| \leq 1$.

$$\min J = \int_{t_0}^{t_1} 1 dt = t_1 - t_0 \quad (80)$$

Hamiltonian:

$$H = 1 + \lambda^T f(x, u) \quad (81)$$

Optimality (Pontryagin's principle):

$$u^*(t) = \arg \max_{|u| \leq 1} H = \arg \max_{|u| \leq 1} \lambda^T f(x, u) \quad (82)$$

For many systems, this yields **bang-bang control**: $u^*(t) \in \{-1, +1\}$ with possible switching points.

i Example: Double Integrator

For $\dot{x}_1 = x_2$, $\dot{x}_2 = u$:

$$H = 1 + \lambda_1 x_2 + \lambda_2 u \quad (83)$$

Optimal control: $u^* = \text{sign}(\lambda_2)$ (bang-bang).

The costate λ_2 evolves deterministically, leading to at most one switch for this problem.

The switching curve divides state space into “apply +1” and “apply -1” regions.

18.4.3 Example 3: LQR as Hamiltonian System

Problem: Quadratic cost with linear dynamics.

$$\min J = \frac{1}{2} x(t_1)^T Q_f x(t_1) + \frac{1}{2} \int_{t_0}^{t_1} [x^T Q x + u^T R u] dt \quad (84)$$

$$\dot{x} = Ax + Bu \quad (85)$$

Hamiltonian:

$$H = \frac{1}{2}(x^T Q x + u^T R u) + \lambda^T (A x + B u) \quad (86)$$

Optimality:

$$u^* = -R^{-1} B^T \lambda \quad (87)$$

Costate equation:

$$\dot{\lambda} = -Q x - A^T \lambda \quad (88)$$

Key insight: Assume $\lambda(t) = P(t)x(t)$ for some symmetric matrix $P(t)$.

Substituting into the costate equation and matching coefficients yields the **Riccati differential equation**:

$$\dot{P} = -Q - A^T P - P A + P B R^{-1} B^T P \quad (89)$$

with terminal condition $P(t_1) = Q_f$.

The optimal control becomes:

$$u^*(t) = -R^{-1} B^T P(t)x(t) = -K(t)x(t) \quad (90)$$

This is the standard LQR result, derived from the Hamiltonian formulation!

! Connection to Tangent Space Framework

LQR succeeds because:

1. Local linearization $(Ax + Bu)$ is exact in tangent space
2. Quadratic cost aligns with second-order Taylor expansion
3. Riccati equation propagates cost-to-go exactly along tangent directions
4. Optimal control is linear feedback exploiting exact local superposition

18.5 Constrained Optimization

18.5.1 Inequality Constraints

Consider control bounds $u_{\min} \leq u \leq u_{\max}$.

Optimality condition (Karush-Kuhn-Tucker):

$$\frac{\partial H}{\partial u} + \mu_{\min} - \mu_{\max} = 0 \quad (91)$$

with complementarity conditions:

$$\begin{aligned} \mu_{\min} &\geq 0, & u - u_{\min} &\geq 0, & \mu_{\min}(u - u_{\min}) &= 0 \\ \mu_{\max} &\geq 0, & u_{\max} - u &\geq 0, & \mu_{\max}(u_{\max} - u) &= 0 \end{aligned} \quad (92)$$

This yields **saturated control**: $u^* = \text{sat}_{[u_{\min}, u_{\max}]}(-R^{-1}B^T\lambda)$ for quadratic Hamiltonians.

18.5.2 State Constraints

Path constraints $g(x, t) \leq 0$ require augmenting the Hamiltonian:

$$H_{\text{aug}} = H + \nu^T g(x, t) \quad (93)$$

where $\nu(t) \geq 0$ are time-varying multipliers (active only when $g = 0$).

The costate equation becomes:

$$\dot{\lambda} = -\frac{\partial H}{\partial x} - \nu^T \frac{\partial g}{\partial x} \quad (94)$$

Handling active constraints requires careful analysis of **boundary arcs** and **junction conditions**—a rich topic beyond this scope (see Bryson & Ho, 1975).

18.6 Summary: Hamiltonian Framework

The Hamiltonian formulation unifies:

- **Forward dynamics:** $\dot{x} = f(x, u)$ (physics)
- **Backward sensitivity:** $\dot{\lambda} = -\frac{\partial H}{\partial x}$ (value propagation)
- **Optimality:** $\frac{\partial H}{\partial u} = 0$ or Pontryagin maximum (control law)

This **three-way duality** is the cornerstone of optimal control theory. It connects directly to the tangent space framework:

1. Hamiltonian defines exact local cost structure
2. Costate transports sensitivity through tangent spaces
3. Optimal control exploits exact local linearity

The next chapter translates these continuous-time ideas into discrete numerical algorithms.

19 Numerical Implementation and Discrete-Time Systems

19.1 Discrete-Time Dynamics

19.1.1 Euler Discretization

For implementation, continuous dynamics are approximated via discrete-time steps. Using Euler's method:

$$x_{k+1} = x_k + f(x_k, u_k)\Delta t + O(\Delta t^2) \quad (95)$$

where $\Delta t = t_{k+1} - t_k$ is the timestep.

19.1.2 Discrete Linearization

At each timestep, linearize:

$$x_{k+1} \approx x_k + [A_k \delta x_k + B_k \delta u_k] \Delta t \quad (96)$$

where:

$$A_k = \frac{\partial f}{\partial x}(x_k, u_k), \quad B_k = \frac{\partial f}{\partial u}(x_k, u_k) \quad (97)$$

19.1.3 Per-Step Superposition

Within each timestep, infinitesimal contributions add exactly:

$$\delta x_{k+1} = \delta x_k + [A_k \delta x_k + B_k \delta u_k] \Delta t \quad (98)$$

This is discrete-time integral superposition—each step's contribution accumulates linearly (up to $O(\Delta t^2)$ residuals).

19.2 Discrete State Transition

19.2.1 Recursive Computation

The discrete state transition operator:

$$\Phi_{k+1|k} = I + A_k \Delta t + O(\Delta t^2) \quad (99)$$

Chaining over multiple steps:

$$\Phi_{N|0} = \Phi_{N|N-1} \Phi_{N-1|N-2} \cdots \Phi_{1|0} \quad (100)$$

This accumulates the linear transport through all timesteps.

19.3 Higher-Order Integration Schemes

19.3.1 Runge-Kutta Methods

Euler's method is first-order accurate ($O(\Delta t)$ local error). For better accuracy, use **Runge-Kutta** methods.

RK4 (Fourth-Order Runge-Kutta):

$$\begin{aligned} k_1 &= f(x_k, u_k) \\ k_2 &= f\left(x_k + \frac{\Delta t}{2} k_1, u_k\right) \\ k_3 &= f\left(x_k + \frac{\Delta t}{2} k_2, u_k\right) \\ k_4 &= f(x_k + \Delta t k_3, u_k) \\ x_{k+1} &= x_k + \frac{\Delta t}{6} (k_1 + 2k_2 + 2k_3 + k_4) + O(\Delta t^5) \end{aligned} \quad (101)$$

Local truncation error: $O(\Delta t^5)$ (fourth-order method).

Global error: $O(\Delta t^4)$ over fixed time interval.

i Linearization at Sub-Steps

For control applications, RK4 evaluates dynamics at intermediate points. When linearizing for DDP/iLQR, we linearize around the RK4 trajectory, not just the endpoints. This requires careful Jacobian computation accounting for the RK4 structure.

19.3.2 Adaptive Step Size

For systems with varying stiffness or curvature, **adaptive timesteps** improve efficiency.

Embedded Runge-Kutta (e.g., Dormand-Prince 5(4)):

- Compute two estimates: $x_{k+1}^{(4)}$ (4th order) and $x_{k+1}^{(5)}$ (5th order)
- Error estimate: $e_k = \|x_{k+1}^{(5)} - x_{k+1}^{(4)}\|$
- Adjust step: If $e_k > \epsilon_{\text{tol}}$, reduce Δt ; if $e_k \ll \epsilon_{\text{tol}}$, increase Δt

Step size update:

$$\Delta t_{\text{new}} = \Delta t_{\text{old}} \left(\frac{\epsilon_{\text{tol}}}{e_k} \right)^{1/5} \cdot \text{safety factor} \quad (102)$$

This concentrates computational effort where curvature is high (large e_k) and relaxes where dynamics are smooth.

19.3.3 Implicit Methods

For **stiff systems** (widely separated timescales), explicit methods require tiny timesteps. **Implicit methods** allow larger steps at the cost of solving nonlinear equations per step.

Backward Euler:

$$x_{k+1} = x_k + \Delta t f(x_{k+1}, u_k) \quad (103)$$

Requires solving $x_{k+1} - x_k - \Delta t f(x_{k+1}, u_k) = 0$ (nonlinear system) via Newton iteration.

Trapezoidal Rule (second-order implicit):

$$x_{k+1} = x_k + \frac{\Delta t}{2} [f(x_k, u_k) + f(x_{k+1}, u_k)] \quad (104)$$

Trade-off: Stability vs. computational cost per step.

19.4 Convergence Analysis

19.4.1 Local vs. Global Error

Local truncation error (LTE): Error in single step assuming exact x_k .

Global error: Accumulated error over many steps.

For a method with $\text{LTE} = O(\Delta t^{p+1})$:

- **Consistency:** $\text{LTE} \rightarrow 0$ as $\Delta t \rightarrow 0$
- **Stability:** Errors don't grow exponentially
- **Convergence:** Global error $= O(\Delta t^p)$ (Lax Equivalence Theorem)

19.4.2 Theorem: Convergence of Euler's Method

! Convergence Theorem

For $\dot{x} = f(x, u)$ with Lipschitz continuous f :

$$\|f(x_1, u) - f(x_2, u)\| \leq L\|x_1 - x_2\| \quad (105)$$

Euler's method converges with global error:

$$\|x(t_N) - x_N\| \leq \frac{M}{2L}(e^{L(t_N - t_0)} - 1)\Delta t \quad (106)$$

where M bounds the second derivative: $\|\frac{d^2x}{dt^2}\| \leq M$.

Proof sketch:

1. LTE at each step: $\|e_k\| \leq \frac{M}{2}\Delta t^2$
2. Error propagation: $e_{k+1} \leq (1 + L\Delta t)e_k + \frac{M}{2}\Delta t^2$
3. Sum geometric series with $N = (t_N - t_0)/\Delta t$ steps
4. Result: $O(\Delta t)$ global convergence $\square$

19.4.3 Higher-Order Methods

- **RK2:** $O(\Delta t^2)$ global error
- **RK4:** $O(\Delta t^4)$ global error
- **RK45** (adaptive): $O(\Delta t^5)$ with automatic step control

Practical rule: Use RK4 for most control problems; adaptive RK45 for high-accuracy requirements or stiff systems.

19.5 Numerical Stability

19.5.1 Stability Region

A method is **absolutely stable** for $\dot{x} = \lambda x$ (test equation) if $|x_{k+1}/x_k| \leq 1$ for all k .

Euler's method stability region: $|1 + \lambda\Delta t| \leq 1$, i.e., $\lambda\Delta t$ must lie in disk of radius 1 centered at -1 in complex plane.

RK4 stability region: Larger (includes larger negative real values), allowing bigger timesteps for stable systems.

Implicit methods (e.g., Backward Euler): *A*-stable—stable for all $\text{Re}(\lambda) < 0$, ideal for stiff systems.

19.5.2 Stiffness and Timestep Selection

A system is **stiff** if it contains dynamics at vastly different timescales. Example:

$$\begin{aligned}\dot{x}_1 &= -0.01x_1 \quad (\text{slow}) \\ \dot{x}_2 &= -1000x_2 \quad (\text{fast})\end{aligned}\tag{107}$$

Explicit methods require $\Delta t \lesssim 2/1000 = 0.002$ (dictated by fast mode), even if we only care about slow dynamics.

Solution: Implicit methods or **exponential integrators** that analytically handle fast modes.

19.6 Error Accumulation and Bounds

19.6.1 Accumulation Over Horizon

For trajectory optimization over N steps:

Total error:

$$\|x(t_N) - x_N\| \leq N \cdot O(\Delta t^{p+1}) \cdot e^{LN\Delta t}\tag{108}$$

where p is the method order.

With $N\Delta t = T$ (fixed horizon):

$$\|x(T) - x_N\| = O(\Delta t^p)\tag{109}$$

Takeaway: Higher-order methods dramatically reduce error for fixed computational budget.

19.6.2 Discrete Tangent Space Errors

At each step, the discrete linearization introduces $O(\Delta t^2)$ residuals (from tangent space curvature). Over N steps:

$$\text{Linearization residual} = O(N\Delta t^2) = O(\Delta t) \quad (110)$$

This matches Euler's truncation error—both are $O(\Delta t)$. Using RK4 reduces integration error to $O(\Delta t^4)$, but linearization residuals remain $O(\Delta t)$ unless we re-linearize more frequently.

Implication for iLQR/DDP: Re-linearize at each iteration using updated trajectory; per-iteration error is $O(\Delta t)$, but iterations refine the trajectory.

19.7 Practical Implementation

19.7.1 Pseudo-Code: RK4 Integration

```
def rk4_step(f, x, u, dt):
    """Single RK4 step for dynamics dx/dt = f(x, u)"""
    k1 = f(x, u)
    k2 = f(x + 0.5*dt*k1, u)
    k3 = f(x + 0.5*dt*k2, u)
    k4 = f(x + dt*k3, u)
    x_next = x + (dt/6)*(k1 + 2*k2 + 2*k3 + k4)
    return x_next

def simulate_trajectory(f, x0, u_traj, dt, N):
    """Simulate forward over N steps"""
    x_traj = [x0]
    for k in range(N):
        x_next = rk4_step(f, x_traj[-1], u_traj[k], dt)
        x_traj.append(x_next)
    return np.array(x_traj)
```

19.7.2 Jacobian Computation

For control synthesis, we need:

$$A_k = \frac{\partial f}{\partial x}(x_k, u_k), \quad B_k = \frac{\partial f}{\partial u}(x_k, u_k) \quad (111)$$

Analytical: Derive symbolically (best for simple f).

Finite differences:

$$A_k[:, i] \approx \frac{f(x_k + \epsilon e_i, u_k) - f(x_k, u_k)}{\epsilon} \quad (112)$$

where e_i is the i -th unit vector, $\epsilon \approx \sqrt{\epsilon_{\text{machine}}}$.

Automatic differentiation: Use libraries (JAX, PyTorch, CasADi) for exact derivatives via computational graphs.

```
import jax
import jax.numpy as jnp

def dynamics(x, u):
    """Example: nonlinear dynamics"""
    return jnp.array([x[1], -jnp.sin(x[0]) + u[0]])

# Jacobians via autodiff
A = jax.jacfwd(dynamics, argnums=0) # f/ x
B = jax.jacfwd(dynamics, argnums=1) # f/ u

x_k = jnp.array([0.5, 0.0])
u_k = jnp.array([0.1])

A_k = A(x_k, u_k) # Shape: (2, 2)
B_k = B(x_k, u_k) # Shape: (2, 1)
```

19.8 Comparison of Methods

Method	Order	Stability	Cost/Step	Best Use Case
Euler	1	Small region	1 eval	Quick prototyping
RK4	4	Moderate region	4 evals	General-purpose

Method	Order	Stability	Cost/Step	Best Use Case
RK45 (adaptive)	4-5	Moderate	6-7 evals	Variable curvature
Backward Euler	1	A-stable	Solve NL system	Stiff systems
Trapezoidal	2	A-stable	Solve NL system	Stiff + accuracy

Recommendation for control:

- Simulation: RK4 or adaptive RK45
- Optimization (DDP/iLQR): RK4 with analytical Jacobians or autodiff
- Real-time MPC: Euler or RK2 (computational constraints)

19.9 Summary: Discrete Implementation

Key takeaways:

1. **Higher-order methods** (RK4) dramatically improve accuracy for fixed timestep
2. **Adaptive timesteps** concentrate effort where curvature is high
3. **Implicit methods** essential for stiff systems
4. **Convergence** guaranteed under Lipschitz conditions: $O(\Delta t^p)$ for p -th order method
5. **Linearization residuals** $O(\Delta t)$ persist even with RK4 (require iterative refinement)

Discrete-time inherits exact per-step superposition (up to discretization error), enabling efficient trajectory optimization algorithms.

20 Control Synthesis via DDP, iLQR, and MPC

20.1 Differential Dynamic Programming (DDP)

20.1.1 Algorithm Overview

DDP solves nonlinear optimal control via iterated local quadratic approximations:

Algorithm: DDP

1. **Initialize:** Nominal trajectory $(x_0, u_0, \dots, x_N, u_N)$
2. **Backward pass:** For $k = N - 1$ down to 0:
 - Compute cost-to-go quadratic approximation $V_k(x) \approx \text{const} + v_k^T(x - x_k) + \frac{1}{2}(x - x_k)^T V_k(x - x_k)$
 - Optimize locally: $\delta u_k^* = \arg \min Q_k(\delta x_k, \delta u_k)$ where Q_k is local quadratic cost
 - Result: Feedback gain K_k and feedforward term d_k
3. **Forward pass:** Apply $\delta u_k = K_k \delta x_k + d_k$ with line search
4. **Iterate** until convergence

20.1.2 Connection to Integral Superposition

DDP exploits:

- **Local linearization** at each step (tangent space)
- **Local quadratic cost** (second-order expansion)
- **Linear transport** of cost-to-go via state transition

Each iteration refines the nominal trajectory by solving a sequence of **exact local problems** and integrating the results.

20.2 Iterative LQR (iLQR)

iLQR is a variant of DDP with first-order cost expansion (quadratic in δu , linear in δx). The backward pass computes:

$$\delta u_k^* = -K_k \delta x_k + d_k \tag{113}$$

where K_k and d_k come from solving local LQR problems along the trajectory.

The key insight: **each local LQR problem is solved exactly** (via Riccati recursion), and the global solution emerges from integrating these exact local solutions.

20.3 Model Predictive Control (MPC)

MPC applies the same principle in real-time:

1. At current state $x(t)$, solve optimal control over finite horizon $[t, t + T]$
2. Apply only the first control $u^*(t)$
3. Measure new state $x(t + \Delta t)$
4. Repeat with updated state and shifted horizon

MPC repeatedly solves a finite-horizon problem and applies its first action. DDP or iLQR can serve as nonlinear subproblem solvers; linear-quadratic MPC often uses a quadratic program instead.

20.4 Detailed DDP Derivation

20.4.1 Cost-to-Go Recursion

Define the **cost-to-go** from step k :

$$V_k(x_k) = \min_{u_k, \dots, u_{N-1}} \left[\phi(x_N) + \sum_{j=k}^{N-1} L(x_j, u_j) \right] \quad (114)$$

subject to $x_{j+1} = f(x_j, u_j)$.

Bellman equation:

$$V_k(x_k) = \min_{u_k} [L(x_k, u_k) + V_{k+1}(f(x_k, u_k))] \quad (115)$$

with $V_N(x_N) = \phi(x_N)$.

20.4.2 Quadratic Approximation

Around nominal trajectory $(\bar{x}_k, \bar{u}_k)$, approximate V_{k+1} as quadratic:

$$V_{k+1}(x) \approx V_{k+1}(\bar{x}_{k+1}) + v_{k+1}^T \delta x_{k+1} + \frac{1}{2} \delta x_{k+1}^T P_{k+1} \delta x_{k+1} \quad (116)$$

where $\delta x_{k+1} = x - \bar{x}_{k+1}$.

Linearize dynamics:

$$\delta x_{k+1} = A_k \delta x_k + B_k \delta u_k + o(\|\delta x_k\|, \|\delta u_k\|) \quad (117)$$

where $A_k = \frac{\partial f}{\partial x}|_k$, $B_k = \frac{\partial f}{\partial u}|_k$.

Expand cost:

$$L(x_k, u_k) \approx L(\bar{x}_k, \bar{u}_k) + L_x^T \delta x_k + L_u^T \delta u_k + \frac{1}{2} \delta x_k^T L_{xx} \delta x_k + \delta x_k^T L_{xu} \delta u_k + \frac{1}{2} \delta u_k^T L_{uu} \delta u_k \quad (118)$$

where $L_x = \frac{\partial L}{\partial x}|_k$, $L_u = \frac{\partial L}{\partial u}|_k$, $L_{xx} = \frac{\partial^2 L}{\partial x^2}|_k$, etc.

20.4.3 Action-Value Function (Q-Function)

Define:

$$Q_k(\delta x_k, \delta u_k) = L_k + V_{k+1}(\delta x_{k+1}) \quad (119)$$

Substituting approximations:

$$Q_k(\delta x_k, \delta u_k) \approx q + q_x^T \delta x_k + q_u^T \delta u_k + \frac{1}{2} \delta x_k^T Q_{xx} \delta x_k + \delta x_k^T Q_{xu} \delta u_k + \frac{1}{2} \delta u_k^T Q_{uu} \delta u_k \quad (120)$$

where:

$$\begin{aligned} q_x &= L_x + A_k^T v_{k+1} \\ q_u &= L_u + B_k^T v_{k+1} \\ Q_{xx} &= L_{xx} + A_k^T P_{k+1} A_k + \sum_i (v_{k+1})_i (f_i)_{xx} \\ Q_{xu} &= L_{xu} + A_k^T P_{k+1} B_k + \sum_i (v_{k+1})_i (f_i)_{xu} \\ Q_{uu} &= L_{uu} + B_k^T P_{k+1} B_k + \sum_i (v_{k+1})_i (f_i)_{uu} \end{aligned} \quad (121)$$

20.4.4 Optimal Control Update

Minimize Q_k with respect to δu_k :

$$\frac{\partial Q_k}{\partial \delta u_k} = q_u + Q_{xu}^T \delta x_k + Q_{uu} \delta u_k = 0 \quad (122)$$

Solving:

$$\delta u_k^* = -Q_{uu}^{-1}(q_u + Q_{xu}^T \delta x_k) = K_k \delta x_k + d_k \quad (123)$$

where:

$$K_k = -Q_{uu}^{-1}Q_{xu}^T, \quad d_k = -Q_{uu}^{-1}q_u \quad (124)$$

20.4.5 Value Function Update

Substituting optimal control back:

$$V_k(\delta x_k) = q + (q_x - Q_{xu}Q_{uu}^{-1}q_u)^T \delta x_k - \frac{1}{2}q_u^T Q_{uu}^{-1}q_u + \frac{1}{2}\delta x_k^T (Q_{xx} - Q_{xu}Q_{uu}^{-1}Q_{xu}^T) \delta x_k \quad (125)$$

Thus:

$$v_k = q_x - Q_{xu}Q_{uu}^{-1}q_u, \quad P_k = Q_{xx} - Q_{xu}Q_{uu}^{-1}Q_{xu}^T \quad (126)$$

This is the **discrete-time Riccati recursion**, solved backward from $k = N - 1$ to $k = 0$.

20.5 Complete DDP Algorithm

! DDP Algorithm (Full)

Initialization: Roll out the chosen inputs from the fixed initial state to obtain a dynamically feasible nominal trajectory. Initialize terminal value derivatives from the terminal cost.

Repeat until convergence:

1. **Backward Pass** ($k = N - 1 \rightarrow 0$):

- Compute Jacobians A_k, B_k and, for full DDP, componentwise second deriva-

tives of the discrete transition f_k

- Compute cost derivatives: $L_x, L_u, L_{xx}, L_{xu}, L_{uu}$
- Compute Q-function matrices: $q_x, q_u, Q_{xx}, Q_{xu}, Q_{uu}$ (Eq. {Equation 121})
- Check positive definiteness of Q_{uu} before solving for K_k, d_k ; regularize and recompute the backward pass if needed (see Equation 124)
- Update value function: v_k, P_k (Eq. {Equation 126})

2. Forward Pass with Line Search:

- For $\alpha \in \{1, 0.5, 0.25, \dots\}$:
 - $u_k^{\text{new}} = \bar{u}_k + \alpha d_k + K_k(x_k^{\text{new}} - \bar{x}_k)$
 - Simulate: $x_{k+1}^{\text{new}} = f(x_k^{\text{new}}, u_k^{\text{new}})$
 - If $J^{\text{new}} < J^{\text{old}}$, accept; else reduce α

3. Update: $\{\bar{x}_k, \bar{u}_k\} \leftarrow \{x_k^{\text{new}}, u_k^{\text{new}}\}$

Convergence check: Report stationarity, feasibility, accepted improvement, and step size. A tiny cost change can mean stagnation; reaching the iteration limit is not convergence.

20.6 Iterative LQR (iLQR) Variant

iLQR omits the value-gradient-weighted **dynamics Hessians** in Equation 121. It retains the full quadratic cost model, including state and state-input curvature:

$$Q_{xx} = L_{xx} + A_k^T P_{k+1} A_k, \quad Q_{xu} = L_{xu} + A_k^T P_{k+1} B_k, \quad Q_{uu} = L_{uu} + B_k^T P_{k+1} B_k. \quad (127)$$

Thus iLQR is not a first-order cost approximation. Full DDP and iLQR can differ in local curvature, computational cost, and convergence behavior; neither has a universal speed or robustness advantage.

20.7 Convergence Requires More Than Positive Input Curvature

Positive Q_{uu} makes a particular local control minimization well posed. It does not alone prove convergence of the nonlinear iteration. Local quadratic Newton convergence requires a non-singular reduced Hessian, sufficient derivative regularity, a close initialization, and eventually accepted full steps. DDP convergence results depend on the particular algorithm and assumptions. iLQR's Hessian approximation does not generally give superlinear convergence. See the [worked control chapter](#) for the precise distinction.

Use nonlinear rollout, a sufficient-decrease acceptance rule, bounded line search, regularization, and a declared failure exit. If regularization changes the computed policy, evaluate that actual policy in the quadratic model rather than substituting an unregularized Schur-complement formula indiscriminately. Constraints require a compatible constrained subproblem.

No benchmark package establishes a universal 5–20 iteration count here. Dense Riccati algebra includes state-control cross products as well as matrix factorizations; derivative and rollout costs depend on the model and implementation. Report timings together with dimensions, horizon, tolerances, derivative method, and achieved residuals.

20.8 Constrained MPC Extension

20.8.1 Handling Control Bounds

For $u_{\min} \leq u \leq u_{\max}$:

Projected DDP:

- Compute unconstrained δu_k^*
- Project: $u_k = \text{clamp}(\bar{u}_k + \alpha \delta u_k^*, u_{\min}, u_{\max})$
- Iterate with active-set refinement

Quadratic Penalty:

An input-violation penalty can use $\rho \|\max(0, u - u_{\max})\|^2 + \rho \|\max(0, u_{\min} - u)\|^2$. This is not an augmented Lagrangian, which also uses multiplier terms and updates. A finite penalty need not enforce exact feasibility, and increasing ρ can worsen conditioning. Check constraint residuals explicitly. Likewise, clipping an unconstrained update is not equivalent to solving the box-constrained quadratic subproblem.

20.8.2 State Constraints

For path constraints $g(x_k) \leq 0$:

Barrier methods:

$$L_{\text{aug}}(x, u) = L(x, u) - \mu \sum_i \log(-g_i(x)) \quad (128)$$

A logarithmic barrier requires a strictly feasible interior. Convergence of a sequence as $\mu \rightarrow 0$ requires appropriately solved subproblems and regularity assumptions; it is not a general global-optimality guarantee for a nonlinear problem.

Sequential quadratic programming (SQP): Linearize constraints, solve QP subproblem per iteration.

20.9 Model Predictive Control (MPC)

20.9.1 Receding Horizon Strategy

At each time t :

1. **Measure** current state $x(t)$
2. **Solve** finite-horizon optimal control (via DDP/iLQR):

$$\min_{u(\cdot)} \int_t^{t+T} L(x, u) d\tau + \phi(x(t+T)) \quad (129)$$

3. **Apply** first control $u^*(t)$ (open-loop for Δt)
4. **Shift** horizon to $[t + \Delta t, t + T + \Delta t]$, repeat

Receding-horizon replanning can respond to the following effects, subject to estimation quality, feasibility, and robustness of the chosen design:

- Model mismatch
- Disturbances
- Unmodeled dynamics

Continuous re-planning exploits local exactness at each step.

20.9.2 Stability Guarantees

! MPC Stability Theorem

For the sampled model $x^+ = F(x, u)$, specify a terminal set X_f and terminal controller k_f . Require that k_f respects the constraints, maps X_f into itself, and satisfies

$$\phi(F(x, k_f(x))) - \phi(x) \leq -\ell_d(x, k_f(x)), \quad x \in X_f,$$

where ℓ_d is the stage cost over a sampling interval. With initial feasibility, exact nominal dynamics, and an attained finite-horizon minimum at each step, shifting the previous optimal sequence and appending k_f supplies a feasible next candidate and the value decrease $V_N(x^+) - V_N(x) \leq -\ell_d(x, u^*)$. A suboptimal solver needs a compatible certified-decrease argument; a small optimization residual alone does not prove this inequality.

Positive-definite state-cost lower bounds, appropriate local upper bounds and continuity of V_N , and admissible bounded trajectories then yield the corresponding stability and attraction result on the certified feasible domain. A nonnegative terminal cost alone is not a Lyapunov certificate, and merely choosing a long horizon does not replace these con-

ditions. Disturbances and model mismatch need a separate robust or practical-stability argument.

20.10 Computational Complexity and Timing Budgets

Dense Riccati algebra involves terms of order $N(n^3 + n^2m + nm^2 + m^3)$, with opportunities to exploit structure. This excludes model-specific derivative evaluation, full-DDP dynamics Hessians, factorization retries, and constrained subproblems. A forward rollout costs N transition evaluations per line-search trial; a general nonlinear transition does not universally cost $O(n)$. Finite differences and automatic differentiation also have different costs depending on the computational graph.

At a control rate of 100 Hz, the available sampling interval is 10 ms. That arithmetic is a budget, not evidence that a specified nonlinear optimization meets it. Record hardware, horizon, dimensions, derivative method, worst-case latency, feasibility, and achieved optimization residuals. Warm starts and iteration limits can reduce runtime, but a time limit requires an admissible fallback policy and does not certify convergence. No universal iteration count or real-time benchmark is established here.

20.11 Robust and Stochastic Extensions

20.11.1 Robust MPC

For uncertain dynamics $\dot{x} = f(x, u, w)$ with bounded $\|w\| \leq \epsilon$:

Min-max formulation:

$$\min_{u(\cdot)} \max_{\|w\| \leq \epsilon} J(x, u, w) \quad (130)$$

Tube MPC: Compute nominal trajectory + error tube; ensure constraints satisfied over tube.

20.11.2 Stochastic MPC

For an Itô model $dx = f(x, u)dt + \sigma(x, u)dW_t$ with Brownian motion W_t , admissible controls must be adapted to the available information. One possible risk-sensitive objective is:

Risk-sensitive cost:

$$\min_{u(\cdot)} \mathbb{E} \left[\int_0^T L(x, u) dt + \phi(x(T)) \right] + \lambda \text{Var}[J] \quad (131)$$

Certainty equivalence has precise validity in standard linear-quadratic-Gaussian settings with their separation assumptions. It is not a general robustness theorem for a nonlinear plant. In general, $\mathbb{E}[f(x, u)] \neq f(\mathbb{E}[x], \mathbb{E}[u])$, and even small noise can matter near constraints, unstable modes, or mode transitions. Evaluate the stochastic or robust formulation actually being used.

20.12 Summary: DDP/iLQR/MPC

Key insights:

1. **DDP/iLQR** solve local quadratic subproblems whose relation to the nonlinear objective depends on the retained derivatives
2. **Riccati recursion** propagates cost-to-go backward through tangent spaces
3. **Feedback gains** K_k exploit exact local superposition
4. **MPC** replans; robustness requires additional design assumptions and verification
5. **Real-time feasibility** depends on measured computation, problem structure, and an admissible fallback

These methods use linear algebra and local derivatives in different ways. Their convergence, stability, and performance require the separate assumptions stated above.

21 Case Studies and Worked Examples

This chapter presents three detailed case studies demonstrating the tangent space framework in practice: spacecraft rendezvous (orbital mechanics), planar robot arm (manipulation), and quadrotor stabilization (underactuated flight). Each example provides an illustrative problem setup and algorithm sketch; the status notice below governs the unvalidated numerical targets.

! Illustrative, Not Validated

The code blocks in this chapter are abbreviated **sketches** (some solver bodies are stubbed), and the numeric “Results” below are **representative, hand-chosen values** illustrating the expected qualitative behavior — they are **not** the output of a validated simulation and have not been benchmarked against empirical data. Treat them as worked-example targets, not measured results.

21.1 Case Study 1: Spacecraft Rendezvous

21.1.1 Problem Formulation

Scenario: Transfer a spacecraft from initial orbit to rendezvous with target (e.g., ISS docking).

Clohessy-Wiltshire (CW) Equations (linearized relative motion):

$$\begin{aligned}\ddot{x} - 2n\dot{y} - 3n^2x &= u_x/m \\ \ddot{y} + 2n\dot{x} &= u_y/m \\ \ddot{z} + n^2z &= u_z/m\end{aligned}\tag{132}$$

where:

- (x, y, z) : relative position (radial, along-track, out-of-plane)
- (u_x, u_y, u_z) : thrust forces
- $n = \sqrt{\mu/a^3}$: orbital rate (mean motion)
- m : spacecraft mass

State vector: $x = [x, y, z, \dot{x}, \dot{y}, \dot{z}]^T \in \mathbb{R}^6$

Control: $u = [u_x, u_y, u_z]^T \in \mathbb{R}^3$

Objective: Minimize fuel consumption while achieving rendezvous.

$$J = \frac{1}{2}x(t_f)^T Q_f x(t_f) + \frac{1}{2} \int_0^{t_f} u^T R u dt \quad (133)$$

with $Q_f = \text{diag}(10^4, 10^4, 10^4, 10^2, 10^2, 10^2)$ (penalize terminal position/velocity), $R = \text{diag}(1, 1, 1)$.

21.1.2 Dynamics Implementation

```
import numpy as np

def spacecraft_dynamics(x, u, n=0.001):
    """Clohessy-Wiltshire equations
    x = [x, y, z, vx, vy, vz], u = [ux, uy, uz]
    n: orbital rate (rad/s), m: mass (kg)
    """
    m = 100.0 # kg
    pos = x[:3]
    vel = x[3:]

    acc = np.array([
        2*n*vel[1] + 3*n**2*pos[0] + u[0]/m,
        -2*n*vel[0] + u[1]/m,
        -n**2*pos[2] + u[2]/m
    ])

    return np.concatenate([vel, acc])

def spacecraft_jacobians(x, u, n=0.001):
    """Analytical Jacobians for CW equations"""
    m = 100.0

    # A = f/ x
    A = np.array([
        [0, 0, 0, 1, 0, 0],
        [0, 0, 0, 0, 1, 0],
        [0, 0, 0, 0, 0, 1],
        [3*n**2, 0, 0, 0, 2*n, 0],
        [0, 0, 0, -2*n, 0, 0],
        [0, 0, -n**2, 0, 0, 0]
    ])
    ])
```

```

# B = f / u
B = np.zeros((6, 3))
B[3:, :] = np.eye(3) / m

return A, B

```

21.1.3 iLQR Solution

Initial trajectory: Straight-line with constant thrust.

```

def solve_spacecraft_ilqr(x0, xf, T=1000, N=50, max_iter=20):
    """Solve spacecraft rendezvous with iLQR"""
    dt = T / N
    n = 0.001 # Orbital rate for LEO (~6400 km radius)

    # Initialize trajectory (simple linear interpolation)
    X_nom = np.linspace(x0, xf, N+1)
    U_nom = np.zeros((N, 3))

    # Cost matrices
    Qf = np.diag([1e4, 1e4, 1e4, 1e2, 1e2, 1e2])
    R = np.eye(3)

    for iteration in range(max_iter):
        # Backward pass: Riccati recursion
        P = Qf
        K_gains = []
        d_feedforward = []

        for k in range(N-1, -1, -1):
            A, B = spacecraft_jacobians(X_nom[k], U_nom[k], n)

            # Discretize
            A_d = np.eye(6) + A * dt
            B_d = B * dt

            # Q-function
            Q_xx = A_d.T @ P @ A_d
            Q_uu = R + B_d.T @ P @ B_d
            Q_xu = A_d.T @ P @ B_d

```

```

    # Gains
    K = -np.linalg.solve(Q_uu, Q_xu.T)

    # Update P
    P = Q_xx - Q_xu @ np.linalg.solve(Q_uu, Q_xu.T)

    K_gains.append(K)
    d_feedforward.append(np.zeros(3)) # Simplified: no feedforward

K_gains.reverse()

# Forward pass with feedback
X_new = [x0]
U_new = []
for k in range(N):
    dx = X_new[-1] - X_nom[k]
    u = U_nom[k] + K_gains[k] @ dx
    U_new.append(u)

    # RK4 integration
    x_next = rk4_step(lambda x, u: spacecraft_dynamics(x, u, n),
                      X_new[-1], u, dt)
    X_new.append(x_next)

# Check convergence
cost_new = 0.5 * X_new[-1].T @ Qf @ X_new[-1] + \
          0.5 * sum(u.T @ R @ u for u in U_new) * dt

print(f"Iter {iteration}: Cost = {cost_new:.2f}")

# Update nominal
X_nom, U_nom = np.array(X_new), np.array(U_new)

if iteration > 0 and abs(cost_new - cost_old) < 1e-3:
    break
cost_old = cost_new

return X_nom, U_nom

```


21.1.4 Results

Initial conditions:

- Position: $[100, 50, 20]$ meters (ahead, along-track, out-of-plane)
- Velocity: $[0.01, 0.02, 0]$ m/s

Terminal state (achieved): $[-0.02, 0.01, 0.003]$ m, $[0.0001, 0.0002, 0]$ m/s

Convergence: 8 iterations, cost reduced from 5.2×10^6 to 3.1×10^3

Fuel consumption: $\int \|u\| dt = 12.4 \text{ N} \cdot \text{s}$ (equivalent to 0.124 kg propellant at $I_{sp} = 300\text{s}$)

i Tangent Space Perspective

The CW equations are **already linear**, so tangent spaces don't vary—perfect test case! The framework reduces to standard LQR, demonstrating that our approach **subsumes linear control** as a special case where all tangent spaces are identical.

For nonlinear orbital mechanics (elliptical orbits, J2 perturbations), tangent spaces vary significantly, and DDP/iLQR exploits local linearity at each point along the trajectory.

21.2 Case Study 2: Planar Robot Arm

21.2.1 Problem Formulation

Configuration: Two-link planar arm with revolute joints.

States: $q = [\theta_1, \theta_2]$ (joint angles), $\dot{q} = [\dot{\theta}_1, \dot{\theta}_2]$ (angular velocities)

Dynamics (Euler-Lagrange):

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau \quad (134)$$

where:

- $M(q)$: Inertia matrix (configuration-dependent)
- $C(q, \dot{q})$: Coriolis/centrifugal terms
- $G(q)$: Gravity terms
- $\tau = [\tau_1, \tau_2]^T$: Joint torques

Specific form (for $L_1 = L_2 = 1$ m, $m_1 = m_2 = 1$ kg):

$$M(q) = \begin{bmatrix} m_1 L_1^2 + m_2(L_1^2 + L_2^2 + 2L_1 L_2 \cos \theta_2) & m_2(L_2^2 + L_1 L_2 \cos \theta_2) \\ m_2(L_2^2 + L_1 L_2 \cos \theta_2) & m_2 L_2^2 \end{bmatrix} \quad (135)$$

$$C(q, \dot{q}) = \begin{bmatrix} -m_2 L_1 L_2 \sin \theta_2 \dot{\theta}_2 & -m_2 L_1 L_2 \sin \theta_2 (\dot{\theta}_1 + \dot{\theta}_2) \\ m_2 L_1 L_2 \sin \theta_2 \dot{\theta}_1 & 0 \end{bmatrix} \quad (136)$$

$$G(q) = \begin{bmatrix} (m_1 + m_2)g L_1 \sin \theta_1 + m_2 g L_2 \sin(\theta_1 + \theta_2) \\ m_2 g L_2 \sin(\theta_1 + \theta_2) \end{bmatrix} \quad (137)$$

State space form: $x = [q, \dot{q}]^T$, $\dot{x} = [\dot{q}, M(q)^{-1}(\tau - C(q, \dot{q})\dot{q} - G(q))]^T$

Task: Move end-effector from $[0.5, 0.5]$ to $[0.3, 0.8]$ (meters, workspace coords) while minimizing effort.

21.2.2 Implementation

```
def robot_arm_dynamics(x, u):
    """Two-link robot arm dynamics
    x = [ 1,  2,  1,  2], u = [ 1,  2]
    """
    L1, L2 = 1.0, 1.0 # Link lengths (m)
    m1, m2 = 1.0, 1.0 # Link masses (kg)
    g = 9.81

    q, dq = x[:2], x[2:]
    1, 2 = q
    1, 2 = dq

    # Inertia matrix
    c2 = np.cos( 2)
    M11 = m1*L1**2 + m2*(L1**2 + L2**2 + 2*L1*L2*c2)
    M12 = m2*(L2**2 + L1*L2*c2)
    M22 = m2*L2**2
    M = np.array([[M11, M12], [M12, M22]])

    # Coriolis
    s2 = np.sin( 2)
    h = -m2*L1*L2*s2
```

```

C = np.array([[h* 2, h*( 1+ 2)], [-h* 1, 0]])

# Gravity
G = np.array([
    (m1+m2)*g*L1*np.sin( 1) + m2*g*L2*np.sin( 1+ 2),
    m2*g*L2*np.sin( 1+ 2)
])

# Acceleration
ddq = np.linalg.solve(M, u - C @ dq - G)

return np.concatenate([dq, ddq])

def forward_kinematics(q):
    """End-effector position"""
    L1, L2 = 1.0, 1.0
    1, 2 = q
    x = L1*np.cos( 1) + L2*np.cos( 1 + 2)
    y = L1*np.sin( 1) + L2*np.sin( 1 + 2)
    return np.array([x, y])

def inverse_kinematics(target_xy):
    """Compute joint angles for target position (elbow-up solution)"""
    x_target, y_target = target_xy
    L1, L2 = 1.0, 1.0

    # Distance to target
    r = np.sqrt(x_target**2 + y_target**2)

    # Elbow angle (law of cosines)
    cos_2 = (r**2 - L1**2 - L2**2) / (2*L1*L2)
    2 = np.arccos(np.clip(cos_2, -1, 1))

    # Shoulder angle
    = np.arctan2(y_target, x_target)
    = np.arctan2(L2*np.sin( 2), L1 + L2*np.cos( 2))
    1 = -

    return np.array([ 1, 2])

```

21.2.3 DDP Solution

Initial config: $q_0 = [30^\circ, 60^\circ]$ (end-effector at $[0.5, 0.5]$)

Target config: $q_f = \text{IK}([0.3, 0.8]) \approx [80^\circ, 40^\circ]$

Results:

- Convergence: 12 iters
- Smooth trajectory respecting arm dynamics (Coriolis effects visible in joint accelerations)
- Final position error: 1.2 cm (within tolerance)

! Nonlinear Coupling

The inertia matrix $M(q)$ **depends on configuration**—tangent spaces vary! At $\theta_2 = 0$ (straight arm), M is maximal; at $\theta_2 = \pm 90^\circ$, different coupling.

DDP succeeds by:

1. Linearizing at each q_k along trajectory (exact local $M(q_k)$)
2. Propagating cost backward through varying tangent spaces
3. Computing gains K_k that adapt to local geometry

This is **impossible** with global linearization (fixed M), but natural in tangent space framework.

21.3 Case Study 3: Quadrotor Stabilization

21.3.1 Problem Formulation

States: Position $p = [x, y, z]^T$, velocity v , orientation (quaternion q), angular velocity ω
 $x = [p, v, q, \omega]^T \in \mathbb{R}^{13}$ (quaternion has 4 components but 3 DOF)

Controls: Rotor thrust forces $u = [u_1, u_2, u_3, u_4]$ (individual motor commands)

Total thrust: $F_{\text{total}} = \sum_{i=1}^4 u_i$

Torques: $\tau = [L(u_2 - u_4), L(u_3 - u_1), c(u_1 - u_2 + u_3 - u_4)]^T$ (roll, pitch, yaw)

where L is arm length, c is torque coefficient.

Dynamics:

$$\begin{aligned}
\dot{p} &= v \\
\dot{v} &= ge_z + \frac{1}{m}R(q)F_{\text{total}}e_z \\
\dot{q} &= \frac{1}{2}q \otimes \omega \\
\dot{\omega} &= J^{-1}(\tau - \omega \times J\omega)
\end{aligned} \tag{138}$$

where $R(q)$ is rotation matrix from body to world frame, J is inertia tensor.

Task: Stabilize at hover ($p = [0, 0, 1]$ m, $v = 0$, level orientation) from perturbed state.

21.3.2 Linearization Challenge

At hover equilibrium:

- $u_i^* = mg/4$ (equal thrust)
- Small perturbations: $\delta p, \delta v, \delta \theta$ (Euler angles), $\delta \omega$

Tangent space at hover is 12D (position, velocity, attitude, rates). Away from hover, tangent space rotates with orientation—this is where framework shines.

21.3.3 DDP Implementation Sketch

```
def quadrotor_cost(x, u, x_target):
    """Quadratic cost: state deviation + control effort"""
    Q = np.diag([10, 10, 50, # Position (z weighted)
                 1, 1, 1,    # Velocity
                 100, 100, 100, # Orientation (Euler angles from quat)
                 1, 1, 1])    # Angular velocity
    R = 0.1 * np.eye(4)

    dx = x - x_target
    return 0.5 * dx.T @ Q @ dx + 0.5 * u.T @ R @ u

# Run DDP for 50 steps (2 seconds at 40 Hz)
X_opt, U_opt = ddp_solver(quadrotor_dynamics, x0, x_target, N=50)
```

Results:

- Settles to 5 cm radius in 1.5 s

- Smooth control (no oscillations)
- Robust to 20% model uncertainty

i Underactuation and Tangent Space

Quadrotor is **underactuated**: 6 DOF (3 position + 3 orientation), but 4 controls. Cannot independently control all directions instantaneously.

Tangent space insight: Locally, the 4 control inputs span a 4D subspace of the 6D acceleration space. DDP finds sequences exploiting this structure:

- Tilt to generate horizontal force (couple position/attitude)
- Integrate over time to achieve desired position

This coupling is **exact in tangent space** (Jacobian B has rank 4), and DDP/iLQR leverages this to plan feasible trajectories.

21.4 Comparative Analysis

Case Study	Nonlinearity Source	Tangent Space Variation	DDP Benefit
Spacecraft	Coriolis (linear CW)	None (constant)	Reduces to LQR
Robot Arm	Configuration-dependent inertia	High (varies with q)	Essential for convergence
Quadrotor	Rotation matrix, underactuation	Moderate (orientation-dependent)	Enables coupled control

Common thread: All exploit **exact local linearization** + **integral superposition**. Success depends on how much tangent spaces vary, not on “degree of nonlinearity.”

21.5 Reference Implementation Notes

The case studies above are illustrative sketches rather than a released codebase; there is no separate companion repository. A complete implementation would include:

- Python/JAX implementations of all three case studies
- Visualization tools (trajectory plots, phase portraits)
- Jupyter notebooks with step-by-step walkthroughs
- Unit tests and benchmarks

22 Conclusion and Future Directions

23 Summary of Key Results

This thesis has developed a unified geometric framework for understanding nonlinear dynamics through the lens of exact infinitesimal superposition. The central insights are:

23.1 Part I: Local Exactness

1. **Tangent spaces are exactly linear** by construction (Chapter 1)
2. **Linearization captures the exact infinitesimal structure** of smooth dynamics (not an approximation)
3. **Time evolution passes through a continuous family of exact linear snapshots** (Chapter 2)
4. **Local optimal control (LQR, Lyapunov) succeeds by exploiting this exactness** (Chapter 3)
5. **Residuals require a matched evolving baseline** and bounds on nonlinear variations and their transport; measured discrepancies can also contain model error

23.2 Part II: Integral Accumulation

6. **Integration preserves linear superposition of variations** (Chapter 5)
7. **State transition operators transport perturbations linearly** through moving tangent spaces (Chapter 6)
8. **Physical conservation laws (impulse, work) are integral consequences** (Chapter 7)
9. **Global residuals scale quadratically** in perturbation size and linearly in time (Chapter 8)

23.3 Part III: Optimal Control Applications

10. **Hamiltonian structure** unifies forward dynamics, backward sensitivity, and optimality (Chapter 9)
11. **Discrete-time schemes** inherit exact per-step superposition (Chapter 10)
12. **DDP/iLQR/MPC** succeed by solving exact local problems and integrating results (Chapter 11)

24 Conceptual Contributions

24.1 Reframing Linearization

The most significant conceptual contribution is **reframing linearization** from “first-order approximation” to “exact infinitesimal structure.” This shift in perspective:

- Explains why linear techniques work so well for nonlinear systems
- Provides rigorous justification for stability analysis via eigenvalues
- Clarifies the role of residuals as geometric features vs. errors
- Unifies diverse control methods (LQR, gain scheduling, MPC) under a common framework

24.2 The Superposition Spectrum

Nonlinearity does not eliminate superposition—it **relocates** it along a spectrum:

Domain	Superposition	Source
Single tangent space	Exact	Linearity of derivatives
Infinitesimal time	Exact	Limit definition of $\dot{x}$
Integrated variations	Exact (first-order)	Linearity of integration
Finite perturbations	Approximate	Second-order interactions under a smooth endpoint map
Global state space	Generally fails	Tangent spaces vary significantly

24.3 Geometric vs. Analytic Perspectives

This thesis bridges two perspectives:

- **Analytic:** Taylor expansions, $o(\cdot)$ notation, limit arguments
- **Geometric:** Manifolds, tangent bundles, curvature

showing they are two languages for the same mathematical structure. This connection enriches both control theory (typically analytic) and differential geometry (typically abstract) by grounding geometric concepts in engineering applications.

25 Practical Impact

The framework has immediate practical implications:

25.1 Design Methodology

1. **Linearize locally** with confidence (it's exact at each point)
2. **Design linear controllers** (LQR, pole placement) for tangent space
3. **Schedule gains** as operating point changes (following tangent spaces)
4. **Monitor residuals** to detect when nonlinear effects dominate
5. **Switch to nonlinear methods** (DDP, MPC) when residuals grow large

25.2 Analysis Tools

- **Lyapunov stability** via linearization with rigorous local conclusions
- **Sensitivity analysis** via state transition operators
- **Performance bounds** via residual quantification
- **Model validation** via geometric residual vs. modeling error decomposition

25.3 Software Implementation

The discrete-time perspective (Part III) directly informs numerical implementations:

- Per-step exact linearization
- Accumulated variations via recursive state transition
- Real-time residual monitoring
- Adaptive timestep selection based on curvature

26 Limitations and Open Questions

26.1 Scope Limitations

As discussed in Chapter 1, the framework requires C^1 smoothness. Extensions needed for:

- **Discontinuous systems** (switching, impacts, friction) via differential inclusions
- **Hybrid systems** (discrete + continuous) via hybrid automata
- **Constrained systems** (inequality constraints) via complementarity theory
- **Stochastic systems** (noise-driven) via stochastic differential geometry

26.2 Computational Challenges

Part III (control synthesis) needs substantial development:

- **Convergence rates** of DDP/iLQR not fully characterized
- **Stability guarantees** for MPC under model mismatch unclear
- **Computational complexity** for high-dimensional systems prohibitive
- **Real-time implementation** requires efficient approximations

26.3 Fundamental Questions

1. **Can residual bounds be improved?** Current $O(\epsilon^2)$ scaling is crude
2. **What is the role of higher-order geometry?** (Torsion, connection, etc.)
3. **How to extend to infinite-dimensional systems?** (PDEs, delay equations)
4. **What about non-smooth optimization?** (Bang-bang control, L1 costs)

27 Future Work

27.1 Theoretical Extensions

Short-term (1-2 years):

- Sharper residual bounds via mixed variations, transport estimates, and declared metrics
- Explicit connection to Riemannian geometry (metric structure, geodesics)
- Stochastic extension via tangent spaces to Wiener processes
- Hybrid system formulation with jump maps

Long-term (3-5 years):

- Infinite-dimensional extension to PDE-constrained optimization
- Information-geometric perspective (Fisher information on manifolds)
- Quantum-classical correspondence in phase space

27.2 Practical Applications

Immediate (0-1 year):

- Software library implementing state transition operators and residual monitoring
- Worked examples: quadrotor, robot arm, spacecraft (complete Part III)
- Comparison studies: proposed framework vs. standard methods

Near-term (1-3 years):

- Industrial case studies (automotive, aerospace, manufacturing)
- Integration with commercial MPC packages
- Real-time embedded implementations

27.3 Pedagogical Development

- Interactive visualizations of moving tangent spaces
- Open-source course materials based on this framework
- Textbook expanding this thesis with exercises and solutions

28 Final Reflection

This thesis has argued that **nonlinear systems are exactly linear in the instantaneous tangent-space mapping**. This is not mere wordplay—it is a precise mathematical statement with profound implications for control theory, numerical analysis, and geometric mechanics.

The tangent space is not an approximation we impose for convenience. It is the **exact local structure** of smooth dynamics, inherited from the definition of derivatives and limits. Superposition does not disappear in nonlinear systems—it **relocates** from global state space to local tangent spaces, where it holds exactly at each fixed state. The tangent space moves with the state, so trajectories are not additive—but the instantaneous structure is exact.

This perspective transforms how we think about nonlinearity:

- Not as a failure of linear methods
- But as a call to **think geometrically**: trajectories winding through manifolds, tangent spaces providing exact local snapshots, state-dependent dynamics shaping nonlinear coupling

The practical payoff is substantial: it explains why LQR works, why Lyapunov analysis is rigorous, why MPC succeeds—all by recognizing that these methods exploit the **exact local linearity** that exists everywhere in smooth nonlinear systems.

The tangent space framework is not an approximation theory. It is the geometric foundation of nonlinear mechanics.

References

1. Abraham, R., & Marsden, J. E. (1978). *Foundations of Mechanics* (2nd ed.). Benjamin/Cummings.
2. Arnold, V. I. (1989). *Mathematical Methods of Classical Mechanics* (2nd ed.). Springer.
3. Bryson, A. E., & Ho, Y.-C. (1975). *Applied Optimal Control*. Hemisphere Publishing.
4. Jacobson, D. H., & Mayne, D. Q. (1970). *Differential Dynamic Programming*. Elsevier.
5. Khalil, H. K. (2002). *Nonlinear Systems* (3rd ed.). Prentice Hall.
6. Lee, J. M. (2012). *Introduction to Smooth Manifolds* (2nd ed.). Springer.
7. Liberzon, D. (2003). *Switching in Systems and Control*. Birkhäuser.
8. Mayne, D. Q., et al. (2000). Constrained model predictive control: Stability and optimality. *Automatica*, 36(6), 789-814.
9. Sastry, S. (1999). *Nonlinear Systems: Analysis, Stability, and Control*. Springer.
10. Tassa, Y., et al. (2012). Synthesis and stabilization of complex behaviors through online trajectory optimization. In *Proc. IEEE/RSJ IROS* (pp. 4906-4913).

Appendices

29 Appendix a: Mathematical Prerequisites

[To be added: Brief review of manifolds, tangent bundles, differential forms, Lie groups for readers unfamiliar with differential geometry]

30 Appendix B: Proofs and Derivations

This appendix supplies explicit bounds and convergence statements used in the main text.

31 B.1 A Finite-Difference Bound From the Endpoint Map

Let $F(a, b) = x(t_1; \bar{u} + av_1 + bv_2)$ use the same initial state for every pair of amplitudes. Assume a C^2 solution map exists on a rectangle containing $0 \leq a, b \leq \epsilon$. The fundamental theorem of calculus, applied twice componentwise, gives the exact identity

$$r = F(\epsilon, \epsilon) - F(\epsilon, 0) - F(0, \epsilon) + F(0, 0) = \int_0^\epsilon \int_0^\epsilon F_{ab}(a, b) db da.$$

Consequently,

$$\|r\| \leq \epsilon^2 \sup_{a, b \in [0, \epsilon]} \|F_{ab}(a, b)\|.$$

The mixed second-variation equation in the main text computes F_{ab} , using derivatives along each corresponding trajectory. A bound on the state Hessian alone omits mixed state-input terms and nonlinear input effects. For control-affine dynamics $f_0(x) + G(x)u$, the term $DG(x)[\xi]v$ remains even though the pure input Hessian vanishes. Existence of one nominal trajectory is also insufficient: all perturbations used by the bound must remain in the smooth model domain.

32 B.2 Intrinsic Curvature Is a Different Quantity

For a connection ∇ , the curvature identity is

$$R(v, w)\xi = \nabla_v \nabla_w \xi - \nabla_w \nabla_v \xi - \nabla_{[v, w]}\xi.$$

The final Lie-bracket term vanishes for commuting coordinate vector fields, but cannot generally be omitted. Curvature can govern parallel-transport holonomy. That geometric operation is not the same as comparing four forced trajectories of an arbitrary vector field.

The counterexample $\dot{x} = u^2$ lives on the flat Euclidean line, where $R = 0$, yet its input interaction is nonzero. It disproves a general bound of this interaction solely by sectional curvature. Coordinate-dependent field derivatives, a specified connection, and intrinsic manifold curvature must be distinguished before proposing a geometric interpretation.

33 B.3 Integration Error and Linearization Error

For explicit Euler applied to a sufficiently smooth initial-value problem over a fixed finite interval, bounded local truncation error $O(h^2)$ and a suitable Lipschitz stability bound yield global integration error $O(h)$. The constant depends on the interval and growth of perturbations. Lipschitz continuity alone does not supply the higher differentiability needed for every claimed high-order method.

If the integrated model is the first-variation approximation rather than the full nonlinear system, reducing h converges to that approximate model. It does not remove finite-perturbation error. For a single smooth input direction, write $x_\epsilon = \bar{x} + \epsilon\xi + \frac{1}{2}\epsilon^2\eta + o(\epsilon^2)$. A computed first-variation endpoint with a uniform Euler error bound satisfies

$$\|x_\epsilon(t_1) - x_{\text{linear}, h}(t_1)\| \leq C_{\text{disc}}h + \frac{1}{2}\epsilon^2\|\eta(t_1)\| + o(\epsilon^2).$$

The self-variation η includes state, input, and mixed derivatives and their transport. This distinguishes convergence of the numerical solver from fidelity of the linearized physical prediction. Assess the two with separate step-size and perturbation-amplitude refinements.

34 Appendix C: Notation Guide and Glossary

35 Primary Notation

Symbol	Meaning
$x \in \mathbb{R}^n$	State vector
$u \in \mathbb{R}^m$	Control input
$f : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}^n$	Vector field (dynamics)
$A = \frac{\partial f}{\partial x}$	State Jacobian
$B = \frac{\partial f}{\partial u}$	Input Jacobian
$\delta x, \delta u$	Infinitesimal perturbations
$\Phi(t_1, t_0)$	State transition operator
$\lambda(t)$	Costate (adjoint variable)
$H(x, u, \lambda, t)$	Hamiltonian
$L(x, u, t)$	Optimization running cost; distinct from a mechanical Lagrangian
r	Residual (failure of superposition)

36 Glossary

Costate (λ): Adjoint variable in optimal control, representing marginal cost with respect to state.

Fréchet Derivative: Generalization of derivative to vector-valued functions; a linear map.

Jacobian: Matrix of partial derivatives; linearization of a vector field.

Manifold: Space that locally resembles Euclidean space; may be curved globally.

Residual: Deviation from additivity when applying multiple perturbations; $O(\epsilon^2)$ interaction under stated smoothness and fixed-horizon assumptions.

State Transition Operator (Φ): Linear map transporting perturbations forward in time via variational dynamics.

Tangent Space ($T_x \mathcal{M}$): Linear space of velocity vectors at point x on manifold $\mathcal{M}$.

Variational Equation: Linearized dynamics governing perturbation evolution.

37 Appendix D: Code Repository Index

[To be added: Links to GitHub repository with Python/Julia implementations of examples, state transition operators, DDP/iLQR, and visualization tools]

End of Thesis